

TRƯỜNG ĐẠI HỌC PHENIKAA

-----oOo-----



Phát hiện Viêm phổi từ X-quang bằng CNN

Course/Học phần: Xử lý ảnh

Code/Mã học phần: EEE703091

Class/Lớp : N01

Instructor/Giảng viên: TS. Nguyễn Văn Tới

Group/Nhóm : Trương Văn Diệu

HÀ NỘI – 2025

DANH SÁCH THÀNH VIÊN TRONG NHÓM

TT	Họ và tên	Lớp -khóa- ngành	Email	Điện thoại	Mã sinh viên	Mức độ đóng góp (theo thang điểm 10)
1	Trương Văn Diệu	K17_KHMT_1	23017208@st.phenikaa- uni.edu.vn	0983078205	23017208	10

MỤC LỤC

Tóm tắt	4
I. Giới thiệu	4
II. Phát biểu bài toán.....	9
III. Cơ sở lý thuyết.....	17
IV. Nghiên cứu liên quan.....	29
V. Dữ liệu và tiền xử lý.....	33
VI. Phương pháp đề xuất	39
VII. Thiết kế thực nghiệm.....	53
VIII. Kết quả thực nghiệm và đánh giá.....	58
IX. Kết luận và hướng phát triển.....	62
X. Tài liệu tham khảo	65
XI. Phụ lục	66
XII. Lời cảm ơn	68

Tóm tắt

Trong những năm gần đây, xử lý ảnh (Image Processing) đã trở thành một hướng tiếp cận hiệu quả trong lĩnh vực thị giác máy tính y tế, đặc biệt là các bài toán phân tích ảnh X-quang. Bệnh viêm phổi là một trong những bệnh lý phổ biến và nguy hiểm liên quan đến đường hô hấp; việc chẩn đoán sớm dựa trên ảnh X-quang ngực đóng vai trò quan trọng trong điều trị và giảm tỷ lệ biến chứng. Tuy nhiên, việc đọc và phân tích ảnh X-quang đòi hỏi bác sĩ có nhiều kinh nghiệm và có thể bị ảnh hưởng bởi các yếu tố như nhiễu ảnh, chất lượng ảnh không đồng đều hoặc biểu hiện bệnh lý không rõ ràng.

Trong báo cáo này, nhóm đề xuất một hệ thống phân loại ảnh X-quang ngực sử dụng mạng nơ-ron tích chập (Convolutional Neural Network – CNN) được xây dựng và huấn luyện bằng thư viện TensorFlow/Keras. Dữ liệu được chuẩn hóa về kích thước 180×180 và chia thành ba tập train/validation/test nhằm đảm bảo quá trình huấn luyện và đánh giá khách quan. Kiến trúc mô hình gồm ba lớp tích chập (Conv2D) kết hợp MaxPooling để tự động trích xuất đặc trưng, sau đó đưa qua các lớp fully-connected để thực hiện phân loại hai nhãn. Mô hình được huấn luyện trong 10 epochs và sử dụng các chỉ số đánh giá như Accuracy, Confusion Matrix và Classification Report.

Kết quả thực nghiệm cho thấy mô hình CNN đạt *độ chính xác trên tập test khoảng 85%*, chứng minh khả năng ứng dụng của học sâu trong việc hỗ trợ nhận diện bệnh lý từ ảnh X-quang. Đồng thời, báo cáo cũng phân tích các yếu tố ảnh hưởng tới hiệu năng và đưa ra hướng phát triển nhằm nâng cao chất lượng mô hình trong tương lai.

I. Giới thiệu

1.1. Bối cảnh và động lực

Trong kỷ nguyên chuyển đổi số, lĩnh vực y tế đang chứng kiến sự phát triển mạnh mẽ của các hệ thống hỗ trợ chẩn đoán dựa trên trí tuệ nhân tạo (AI). Đặc biệt, với sự bùng nổ của dữ liệu ảnh y khoa như X-quang, CT, MRI..., việc ứng dụng học máy và học sâu để tự động phân tích hình ảnh đã trở thành một hướng nghiên cứu có giá trị thực tiễn cao. Trong đó, *ảnh chụp X-quang ngực* là một trong những loại dữ liệu phổ biến nhất nhờ chi phí thấp, khả năng triển khai rộng rãi và hỗ trợ hiệu quả trong sàng lọc nhiều bệnh lý liên quan đến phổi.

Viêm phổi (Pneumonia) là một bệnh lý đường hô hấp thường gặp, có thể xuất hiện ở nhiều độ tuổi và đặc biệt nguy hiểm đối với trẻ nhỏ, người già hoặc người có hệ miễn dịch yếu. Nếu không được phát hiện và điều trị kịp thời, viêm phổi có thể gây ra các biến chứng nghiêm trọng, thậm chí ảnh hưởng tới tính mạng. Trong thực tế lâm sàng, ảnh X-quang ngực thường được sử dụng như một phương pháp quan trọng để phát hiện và theo dõi tình trạng viêm phổi. Tuy nhiên, quá trình đọc ảnh và đưa ra kết luận phụ thuộc nhiều vào chuyên môn và kinh nghiệm của bác sĩ, đồng thời có thể bị ảnh hưởng bởi các yếu tố như:

Chất lượng ảnh không đồng đều (nhiều, mờ, thiếu sáng, độ tương phản thấp).

Biểu hiện bệnh lý không rõ ràng hoặc dễ nhầm lẫn với các bệnh khác.

Sự quá tải tại các cơ sở y tế khiến thời gian phân tích ảnh bị rút ngắn.

Những thách thức trên đặt ra nhu cầu cần có một hệ thống hỗ trợ tự động nhằm *tăng tốc quá trình sàng lọc, giảm sai sót và hỗ trợ bác sĩ đưa ra quyết định chính xác hơn.*

Trong những năm gần đây, *mạng nơ-ron tích chập (Convolutional Neural Network – CNN)* đã chứng minh hiệu quả vượt trội trong các bài toán thị giác máy tính nhờ khả năng tự động học đặc trưng trực tiếp từ dữ liệu ảnh. CNN không cần thiết kế đặc trưng thủ công mà có thể rút trích đặc trưng phân cấp từ đơn giản đến phức tạp thông qua các lớp tích chập và pooling. Nhờ đó, CNN được ứng dụng rộng rãi trong nhiều bài toán liên quan tới ảnh y khoa như phát hiện khối u, nhận dạng tổn thương và phân loại bệnh lý.

Xuất phát từ bối cảnh trên, báo cáo lựa chọn đề tài *phân loại ảnh X-quang ngực (người/không người hoặc tương ứng 2 lớp trong bộ dữ liệu)* bằng phương pháp học sâu nhằm xây dựng một mô hình có khả năng dự đoán nhãn ảnh dựa trên dữ liệu huấn luyện. Đây là một hướng tiếp cận vừa có giá trị học thuật trong môn Học sâu, vừa có ý nghĩa thực tiễn vì có thể làm nền tảng để phát triển các hệ thống hỗ trợ chẩn đoán trong tương lai.

1.2. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng và đánh giá một mô hình học sâu có khả năng *phân loại ảnh X-quang ngực thành hai lớp* dựa trên dữ liệu huấn luyện. Thông qua đó, mô hình có thể đóng vai trò như một công cụ hỗ trợ tự động trong các hệ thống phân tích ảnh y khoa, góp phần nâng cao hiệu quả sàng lọc và giảm phụ thuộc hoàn toàn vào việc quan sát thủ công.

a) Mục tiêu tổng quát

Xây dựng một mô hình mạng nơ-ron tích chập (CNN) sử dụng thư viện TensorFlow/Keras để thực hiện phân loại ảnh X-quang ngực theo bài toán phân loại nhị phân (2 lớp).

Thiết lập đầy đủ quy trình xử lý dữ liệu, huấn luyện và đánh giá mô hình nhằm đảm bảo kết quả có độ tin cậy và có thể mở rộng trong tương lai.

b) Mục tiêu cụ thể

Đề tài hướng tới các mục tiêu chi tiết sau:

Chuẩn bị và tổ chức dữ liệu

Mount Google Drive, giải nén bộ dữ liệu ảnh.

Tổ chức dữ liệu thành các tập *Train / Validation / Test* với cấu trúc thư mục hợp lý.

Đọc dữ liệu từ thư mục bằng `tf.keras.utils.image_dataset_from_directory`.

Tiền xử lý dữ liệu và chuẩn hóa đầu vào

Chuẩn hóa kích thước ảnh về 180×180 nhằm đảm bảo thống nhất dữ liệu đầu vào.

Chuẩn hóa giá trị pixel bằng lớp Rescaling(1./255) để đưa ảnh về miền $[0,1]$.

Xây dựng pipeline nhằm tăng hiệu quả huấn luyện (batching, prefetching).

Xây dựng mô hình CNN phân loại

Thiết kế mô hình CNN theo hướng đơn giản và hiệu quả, bao gồm:

- + Các lớp Conv2D và MaxPooling2D để trích xuất đặc trưng ảnh.
- + Các lớp Flatten, Dense để thực hiện phân loại.

Sử dụng lớp đầu ra Dense(2, activation='softmax') cho bài toán phân loại hai lớp.

Huấn luyện mô hình

Thiết lập siêu tham số huấn luyện: số epochs, batch size, optimizer và hàm loss phù hợp.

Huấn luyện mô hình trên tập Train, theo dõi chất lượng mô hình bằng tập Validation.

Đánh giá và phân tích kết quả

Đánh giá hiệu năng mô hình trên tập Test.

Sử dụng các chỉ số đánh giá:

- + Accuracy
- + Confusion Matrix
- + Precision, Recall, F1-score (classification report)

Phân tích các trường hợp dự đoán sai và nguyên nhân.

c) Tiêu chí hoàn thành đề tài

Một mô hình được xem là đạt yêu cầu khi:

Pipeline huấn luyện – đánh giá chạy ổn định, dữ liệu được xử lý đúng định dạng.

Mô hình có thể dự đoán đúng nhãn ảnh X-quang và cho kết quả khả quan trên tập test.

Kết quả được trình bày rõ ràng thông qua các biểu đồ và bảng đánh giá.

Có phân tích, nhận xét ưu/nhược điểm của mô hình và đề xuất hướng cải thiện.

1.3. Các đóng góp của bài báo cáo

Trong phạm vi môn học Học sâu, báo cáo này thực hiện xây dựng và đánh giá mô hình học sâu cho bài toán phân loại ảnh X-quang ngực. Các đóng góp chính của báo cáo bao gồm:

Xây dựng pipeline xử lý dữ liệu ảnh hoàn chỉnh: từ bước đọc dữ liệu từ Google Drive, giải nén, tổ chức thư mục cho đến tạo tập huấn luyện/kiểm định/kiểm tra theo cấu trúc Train/Validation/Test.

Thiết kế và triển khai mô hình CNN bằng thư viện TensorFlow/Keras, bao gồm các lớp tích chập (Conv2D), pooling (MaxPooling) và fully-connected nhằm tự động trích xuất đặc trưng và thực hiện phân loại.

Thực hiện huấn luyện mô hình với cấu hình phù hợp, theo dõi quá trình học thông qua các chỉ số loss/accuracy trên tập train và validation.

Đánh giá khách quan trên tập test với các tiêu chí: Accuracy, Confusion Matrix và Classification Report (Precision, Recall, F1-score).

Phân tích kết quả thực nghiệm: so sánh khả năng phân loại, chỉ ra nguyên nhân dẫn đến dự đoán sai và đề xuất hướng cải thiện chất lượng mô hình trong tương lai (tăng dữ liệu, augmentation, transfer learning, fine-tuning...).

Nhìn chung, báo cáo thể hiện đầy đủ quy trình triển khai một bài toán học sâu từ dữ liệu ảnh thực tế: ***tiền xử lý → xây dựng mô hình → huấn luyện → đánh giá → phân tích.***

1.4. Bố cục báo cáo

Nội dung báo cáo được tổ chức thành các chương/mục như sau:

Chương I – Giới thiệu: Trình bày bối cảnh, động lực lựa chọn đề tài, mục tiêu nghiên cứu và các đóng góp chính của báo cáo.

Chương II – Phát biểu bài toán: Mô tả bài toán phân loại ảnh X-quang, xác định đầu vào/đầu ra, biểu diễn toán học và các thách thức của bài toán.

Chương III – Cơ sở lý thuyết: Tổng hợp các kiến thức nền tảng liên quan đến CNN, hàm mất mát, tối ưu hóa, các kỹ thuật chống overfitting và chỉ số đánh giá.

Chương IV – Nghiên cứu liên quan: Trình bày các phương pháp truyền thống và học sâu trong phân loại ảnh, đặc biệt là các mô hình học chuyển giao được sử dụng rộng rãi.

Chương V – Dữ liệu và tiền xử lý: Mô tả bộ dữ liệu, thống kê số lượng, phân bố lớp, quy trình tiền xử lý và tăng cường dữ liệu.

Chương VI – Phương pháp đề xuất: Trình bày kiến trúc mô hình CNN, pipeline huấn luyện bằng thư viện, các chiến lược huấn luyện và minh họa mô hình.

Chương VII – Thiết kế thực nghiệm: Nêu rõ môi trường thực nghiệm, cấu hình huấn luyện, protocol đánh giá và tính tái lập.

Chương VIII – Kết quả thực nghiệm và đánh giá: Tổng hợp kết quả huấn luyện, kết quả test, bảng so sánh mô hình và phân tích lỗi.

Chương IX – Thí nghiệm mở rộng (nếu có): Khảo sát ảnh hưởng của các yếu tố như augmentation, fine-tuning, optimizer...

Chương X – Ứng dụng minh họa: Trình bày quy trình suy luận và demo dự đoán ảnh (có thể triển khai Streamlit/Gradio).

Chương XI – Kết luận và hướng phát triển: Tổng kết kết quả đạt được, hạn chế và định hướng cải tiến trong tương lai.

Tài liệu tham khảo, Phụ lục và Lời cảm ơn được trình bày ở cuối báo cáo

II. Phát biểu bài toán

2.1. Mô tả bài toán

Bài toán trong báo cáo này là *bài toán phân loại ảnh (Image Classification)* sử dụng học sâu, với mục tiêu xây dựng mô hình có khả năng *tự động nhận biết và phân loại ảnh X-quang ngực theo hai lớp* dựa trên dữ liệu huấn luyện. Cụ thể, với mỗi ảnh X-quang đầu vào, hệ thống sẽ dự đoán ảnh thuộc lớp nào trong hai lớp đã được gán nhãn sẵn trong bộ dữ liệu.

Trong quy trình thực tế, ảnh X-quang ngực có thể chứa nhiều đặc trưng quan trọng liên quan đến cấu trúc phổi, vùng tổn thương, độ mờ, vùng đông đặc... Những đặc trưng này thường được bác sĩ quan sát và phân tích để đưa ra chẩn đoán. Tuy nhiên, việc quan sát thủ công có thể bị ảnh hưởng bởi nhiều yếu tố như kinh nghiệm của người đọc ảnh, chất lượng ảnh không đồng đều hoặc số lượng ảnh cần xử lý lớn. Do đó, việc xây dựng một mô hình học sâu có thể tự động trích xuất đặc trưng và dự đoán nhãn là cần thiết và có ý nghĩa.

Bài toán được giải quyết bằng cách sử dụng *mạng nơ-ron tích chập (Convolutional Neural Network – CNN)*, một kiến trúc nổi bật trong thị giác máy tính. Mô hình CNN có khả năng học đặc trưng trực tiếp từ dữ liệu ảnh thông qua các lớp tích chập và pooling, từ đó biểu diễn ảnh thành các đặc trưng trừu tượng và thực hiện phân loại.

Đầu vào và đầu ra của bài toán

Đầu vào (Input): Ảnh X-quang ngực đã được chuẩn hóa về kích thước 180×180 (3 kênh màu) và chuẩn hóa giá trị pixel về miền $[0, 1]$.

Đầu ra (Output): Nhãn dự đoán thuộc một trong *hai lớp* (nhị phân), tương ứng với cấu trúc nhãn trong bộ dữ liệu.

Dạng bài toán

Về bản chất, đây là bài toán:

Phân loại ảnh có giám sát (Supervised Learning)

Phân loại nhị phân (Binary Classification) theo 2 nhãn

Thực hiện dự đoán dựa trên dữ liệu đã gán nhãn trong bộ dữ liệu train

Thông qua quá trình học từ dữ liệu, mô hình CNN sẽ tìm ra mối liên hệ giữa đặc trưng ảnh và nhãn lớp, từ đó có thể dự đoán cho các ảnh mới chưa từng xuất hiện trong quá trình huấn luyện.

2.2. Đầu vào – đầu ra của hệ thống

Hệ thống đề xuất trong báo cáo là một mô hình học sâu có khả năng nhận ảnh X-quang ngực làm đầu vào và trả về nhãn phân loại tương ứng. Toàn bộ hệ thống có thể được mô tả dưới dạng một pipeline gồm các bước: **tiền xử lý dữ liệu** → **trích xuất đặc trưng bằng CNN** → **phân loại** → **trả kết quả dự đoán**.

a) Đầu vào của hệ thống (Input)

Đầu vào của hệ thống là **ảnh X-quang ngực** được thu thập từ bộ dữ liệu ảnh và đã được xử lý để phù hợp với mô hình CNN.

Cụ thể, đầu vào có các đặc điểm sau:

Dạng dữ liệu: ảnh số (image tensor)

Kích thước ảnh: được chuẩn hóa về **180×180**

Số kênh màu: 3 kênh (RGB) theo thiết lập mặc định khi đọc ảnh bằng `image_dataset_from_directory`

Giá trị pixel: được chuẩn hóa về miền **[0, 1]** nhờ lớp Rescaling (1./255)

Batch input: các ảnh được đưa vào theo từng batch (mini-batch) nhằm tối ưu tốc độ huấn luyện

Biểu diễn toán học cho một ảnh đầu vào có thể viết dưới dạng tensor:

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C}$$

trong đó:

$H = 180$, $W = 180$ là chiều cao và chiều rộng ảnh

$C = 3$ là số kênh màu

b) Đầu ra của hệ thống (Output)

Đầu ra của hệ thống là *nhãn dự đoán* của ảnh đầu vào thuộc một trong hai lớp phân loại.

Mô hình CNN sử dụng tầng cuối cùng là:

`Dense(2, activation="softmax")`

Do đó, đầu ra của mô hình có dạng một vector xác suất:

$$\hat{\mathbf{y}} = [p_0, p_1]$$

Trong đó:

p_0 là xác suất ảnh thuộc lớp 0

p_1 là xác suất ảnh thuộc lớp 1

$$p_0 + p_1 = 1$$

Sau đó, nhãn dự đoán cuối cùng được xác định bằng:

$$\hat{y} = \arg \max(\hat{\mathbf{y}})$$

Tức là hệ thống chọn lớp có xác suất lớn nhất làm kết quả dự đoán.

c) Mô tả pipeline tổng quát của hệ thống

Quy trình hoạt động của hệ thống có thể mô tả như sau:

Bước 1: Nhập dữ liệu ảnh

đọc ảnh từ thư mục dataset theo cấu trúc train/val/test

Bước 2: Tiền xử lý

resize ảnh về 180×180

chuẩn hóa pixel về $[0,1]$

Bước 3: Trích xuất đặc trưng

sử dụng các lớp Conv2D + MaxPooling để học đặc trưng tự động

Bước 4: Phân loại

Flatten + Dense để đưa ra xác suất thuộc 2 lớp

Bước 5: Trả kết quả

xuất ra nhãn dự đoán và xác suất tương ứng

2.3. Biểu diễn toán học bài toán phân loại

Bài toán trong báo cáo là bài toán *phân loại ảnh có giám sát (supervised image classification)*. Với mỗi ảnh X-quang đầu vào $\mathbf{X}$, mô hình cần dự đoán nhãn tương ứng thuộc một trong hai lớp.

a) Biểu diễn dữ liệu và nhãn

Tập dữ liệu huấn luyện có thể biểu diễn như sau:

$$\mathcal{D} = \{(\mathbf{X}_i, y_i)\}_{i=1}^N$$

Trong đó:

N là số lượng ảnh trong tập huấn luyện.

$\mathbf{X}_i \in \mathbb{R}^{180 \times 180 \times 3}$ là ảnh thứ i sau khi chuẩn hóa.

$y_i \in \{0, 1\}$ là nhãn thật tương ứng với 2 lớp.

b) Mô hình dự đoán

Mạng CNN có thể xem như một hàm ánh xạ tham số hóa bởi bộ trọng số θ :

$$f_{\theta}(\mathbf{X}) = \hat{\mathbf{y}}$$

Trong đó đầu ra $\hat{\mathbf{y}}$ là vector xác suất thuộc hai lớp:

$$\hat{\mathbf{y}} = [\hat{y}_0, \hat{y}_1]$$

Mô hình sử dụng tầng cuối cùng Softmax:

$$\hat{y}_k = \text{Softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=0}^1 e^{z_j}}, \quad k \in \{0, 1\}$$

Ở đây:

$\mathbf{Z} = [z_0, z_1]$ là vector logit (đầu ra trước Softmax) của tầng Dense cuối.

$\hat{y}_k$ là xác suất dự đoán thuộc lớp k .

Nhãn dự đoán cuối cùng được lấy theo xác suất lớn nhất:

$$\hat{c} = \arg \max_k (\hat{y}_k)$$

c) Hàm mất mát (Loss Function)

Với bài toán phân loại nhị phân theo dạng 2 output neuron (Softmax), ta sử dụng *Cross Entropy Loss*.

Nếu nhãn thật được biểu diễn dưới dạng one-hot $y = [y_0, y_1]$, loss của một mẫu là:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=0}^1 y_k \log(\hat{y}_k)$$

Trên toàn bộ tập huấn luyện, hàm mất mát trung bình:

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathbf{y}_i, \hat{\mathbf{y}}_i)$$

Mục tiêu của huấn luyện là tìm tham số θ để hàm mất mát nhỏ nhất:

$$\theta^* = \arg \min_{\theta} \mathcal{J}(\theta)$$

d) Lan truyền ngược và cập nhật tham số

Trong quá trình huấn luyện, mô hình cập nhật trọng số dựa trên gradient của loss theo tham số:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{J}(\theta)$$

Trong đó:

η là learning rate

$\nabla_{\theta} J(\theta)$ là đạo hàm của loss theo trọng số của mạng (tính thông qua backpropagation)

Trong notebook, mô hình được tối ưu bằng **Adam optimizer**. Adam cải tiến từ SGD bằng cách kết hợp moment bậc 1 và 2:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Trong đó $g_t = \nabla_{\theta} J(\theta)$ là gradient tại bước t.

e) *Chỉ số đánh giá*

Sau huấn luyện, mô hình được đánh giá bằng các chỉ số:

Accuracy:

$$Accuracy = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}}$$

Confusion Matrix: thể hiện số lượng dự đoán đúng/sai theo từng lớp.

Precision, Recall, F1-score: đánh giá chi tiết hơn khi dữ liệu mất cân bằng.

2.4. Các thách thức của bài toán

Mặc dù bài toán phân loại ảnh X-quang ngực bằng học sâu có nhiều tiềm năng ứng dụng trong thực tế, tuy nhiên việc xây dựng một mô hình đạt độ chính xác cao và có khả năng tổng quát tốt vẫn gặp nhiều khó khăn. Các thách thức chính của bài toán bao gồm:

a) Chất lượng ảnh X-quang không đồng đều

Ảnh X-quang trong thực tế (và cả trong các bộ dữ liệu công khai) thường có sự khác biệt lớn về:

độ sáng/tối, độ tương phản,

mức độ nhiễu,

độ sắc nét,

chất lượng máy chụp và góc chụp.

Điều này làm cho mô hình CNN khó học được các đặc trưng ổn định và dẫn đến việc dự đoán sai ở những ảnh có chất lượng thấp hoặc bị mờ/nhiều.

b) Sự đa dạng lớn về hình thái bệnh lý

Viêm phổi hoặc các bất thường ở phổi có thể biểu hiện theo nhiều dạng khác nhau:

vùng mờ nhỏ, rải rác,

vùng đông đặc rõ rệt,

tổn thương lan rộng,

hoặc thậm chí biểu hiện khá mờ nhạt.

Trong nhiều trường hợp, biểu hiện bệnh lý không rõ ràng hoặc gần giống các bệnh phổi khác, khiến ngay cả người đọc ảnh cũng có thể gặp khó khăn. Điều này tạo ra thách thức cho mô hình trong việc học và phân biệt chính xác giữa hai lớp.

c) Mất cân bằng dữ liệu (Class Imbalance)

Trong nhiều bộ dữ liệu X-quang, số lượng ảnh giữa các lớp thường *không cân bằng*, ví dụ:

lớp bệnh (Pneumonia) nhiều hơn hoặc ít hơn lớp bình thường (Normal).

Nếu không xử lý tốt, mô hình có xu hướng ưu tiên dự đoán lớp chiếm đa số để đạt accuracy cao, nhưng lại gây giảm hiệu quả khi đánh giá theo precision/recall cho lớp thiểu số. Do đó, cần quan tâm đến các chỉ số như F1-score, confusion matrix, hoặc áp dụng class weight.

d) Nguy cơ overfitting do mô hình học sâu

CNN có khả năng học đặc trưng rất mạnh, nhưng nếu:

dữ liệu huấn luyện không đủ lớn,

augmentation chưa đủ đa dạng,

mô hình có quá nhiều tham số,

thì mô hình dễ rơi vào tình trạng *overfitting*, tức là đạt kết quả cao trên tập train nhưng giảm đáng kể trên validation/test.

Trong bài toán ảnh y khoa, overfitting là vấn đề phổ biến do dữ liệu khó thu thập và thường không phong phú như các bộ dữ liệu ảnh tự nhiên (như CIFAR-10, ImageNet).

e) Sai lệch phân phối dữ liệu (Domain Shift)

Một trong những khó khăn lớn của ảnh y khoa là dữ liệu ở từng nguồn có thể khác nhau:

bệnh viện khác nhau → máy chụp khác nhau,

thông số chụp khác nhau → ảnh có độ tương phản khác,

quy trình tiền xử lý khác nhau.

Điều này dẫn tới hiện tượng domain shift: mô hình học tốt trên dữ liệu A nhưng có thể giảm mạnh hiệu năng khi đưa sang dữ liệu B. Thách thức này ảnh hưởng trực tiếp đến khả năng triển khai mô hình trong thực tế.

f) Yêu cầu tính giải thích (Explainability) trong y tế

Trong lĩnh vực y tế, một mô hình dự đoán không chỉ cần đúng mà còn cần:

giải thích được vì sao mô hình đưa ra kết luận,

minh bạch và đáng tin cậy.

CNN thường bị xem là “hộp đen” (black-box), gây khó khăn khi giải thích nguyên nhân dự đoán. Điều này đặt ra thách thức trong việc nâng cao độ tin cậy, ví dụ dùng Grad-CAM để trực quan hóa vùng chú ý (hướng phát triển).

g) Hạn chế tài nguyên tính toán

Huấn luyện mô hình học sâu trên ảnh thường tốn nhiều tài nguyên:

GPU,

bộ nhớ RAM,

thời gian training.

Đặc biệt với các mô hình sâu hơn hoặc transfer learning, yêu cầu tài nguyên sẽ lớn hơn. Trong môi trường như Google Colab, giới hạn thời gian và tài nguyên cũng ảnh hưởng đến việc lựa chọn kiến trúc, batch size và số epoch.

Tóm lại, bài toán phân loại ảnh X-quang bằng CNN chịu ảnh hưởng bởi nhiều yếu tố: chất lượng dữ liệu, mất cân bằng lớp, overfitting, và khả năng tổng quát hóa sang các tập dữ liệu khác nhau. Do đó, báo cáo cần thiết kế pipeline tiền xử lý hợp lý, lựa chọn mô hình phù hợp và đánh giá bằng nhiều chỉ số thay vì chỉ dựa vào accuracy.

III. Cơ sở lý thuyết

3.1. Tổng quan học sâu trong thị giác máy tính

Trong những năm gần đây, *học sâu (Deep Learning)* đã trở thành một lĩnh vực trọng tâm của trí tuệ nhân tạo, đặc biệt hiệu quả trong các bài toán xử lý dữ liệu phi cấu trúc như ảnh, âm thanh và văn bản. Khác với các phương pháp học máy truyền thống (yêu cầu trích xuất đặc trưng thủ công), học sâu có khả năng *tự động học đặc trưng (feature learning)* từ dữ liệu thông qua các tầng biểu diễn (layers).

Trong thị giác máy tính, học sâu đã tạo ra bước tiến lớn trong nhiều nhiệm vụ quan trọng như:

phân loại ảnh (image classification),

phát hiện đối tượng (object detection),

phân đoạn ảnh (image segmentation),

nhận diện khuôn mặt, nhận diện chữ viết tay,...

Một lý do quan trọng khiến học sâu thành công là nhờ các mô hình có thể học đặc trưng theo dạng phân cấp:

Các tầng thấp học đặc trưng đơn giản (cạnh, góc, texture).

Các tầng cao học đặc trưng phức tạp hơn (hình dạng, cấu trúc vật thể).

Tầng cuối cùng đưa ra dự đoán nhãn.

Đối với ảnh y khoa nói chung và ảnh X-quang ngực nói riêng, học sâu đóng vai trò quan trọng do:

dữ liệu ảnh chứa nhiều thông tin ẩn khó biểu diễn bằng đặc trưng thủ công,

mô hình có khả năng học trực tiếp các dấu hiệu bệnh lý trên ảnh,

hỗ trợ tăng hiệu quả sàng lọc và giảm áp lực cho nhân viên y tế.

Một bài toán phổ biến trong ảnh y khoa là *phân loại ảnh (classification)* để xác định ảnh thuộc lớp bệnh hoặc bình thường. Trong báo cáo này, hệ thống được xây dựng nhằm giải quyết bài toán phân loại nhị phân dựa trên ảnh X-quang ngực.

3.2. Mạng nơ-ron tích chập (CNN)

CNN (Convolutional Neural Network) là một kiến trúc học sâu được thiết kế tối ưu cho dữ liệu ảnh. Thay vì phải trích xuất đặc trưng thủ công như các phương pháp truyền thống, CNN có khả năng *tự động học đặc trưng* trực tiếp từ dữ liệu thông qua các lớp tích chập và pooling. Trong bài toán phân loại ảnh X-quang ngực, CNN có thể học được các đặc trưng quan trọng như vùng mờ, vùng đông đặc hoặc sự thay đổi kết cấu mô phổi, từ đó hỗ trợ dự đoán nhãn ảnh.

a) Tích chập (Convolution)

Lớp tích chập sử dụng các bộ lọc (kernel) kích thước nhỏ (ví dụ 3×3) trượt trên ảnh đầu vào để tạo ra các bản đồ đặc trưng (feature map). Mỗi kernel có nhiệm vụ phát hiện một kiểu đặc trưng nhất định như cạnh, góc, kết cấu.

Biểu diễn tích chập có thể viết:

$$\mathbf{F}(i, j) = \sum_m \sum_n \mathbf{K}(m, n) \cdot \mathbf{X}(i + m, j + n)$$

Trong đó:

X là ảnh đầu vào,

K là kernel,

F là feature map đầu ra.

Sau tích chập thường áp dụng hàm kích hoạt **ReLU** để tăng tính phi tuyến:

$$ReLU(x) = \max(0, x)$$

b) Pooling

Pooling (thường là MaxPooling) là lớp dùng để *giảm kích thước* feature map, giảm số tham số và tăng khả năng chống nhiễu. MaxPooling lấy giá trị lớn nhất trong mỗi vùng (ví dụ 2×2):

$$P(i, j) = \max_{(m, n) \in \Omega} F(i + m, j + n)$$

Nhờ đó mô hình vừa giảm độ phức tạp vừa giữ lại các đặc trưng quan trọng nhất.

c) Flatten, Fully Connected

Sau nhiều lớp tích chập và pooling, các feature map được chuyển thành vector 1 chiều bằng lớp *Flatten*:

$$v = \text{Flatten}(F)$$

Vector này sau đó được đưa vào các lớp *Fully Connected (Dense)* để thực hiện phân loại. Tầng Dense cuối cùng thường dùng *Softmax* để đưa ra xác suất mỗi lớp:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Cuối cùng, mô hình dự đoán ảnh thuộc lớp có xác suất lớn nhất.

3.3. Hàm kích hoạt

Trong mạng nơ-ron, *hàm kích hoạt (Activation Function)* đóng vai trò đưa tính *phi tuyến* vào mô hình. Nếu không có hàm kích hoạt, mạng nơ-ron dù có nhiều lớp cũng chỉ tương đương một phép biến đổi tuyến tính, không thể học được các quan hệ phức tạp trong dữ liệu ảnh. Trong CNN, hàm kích hoạt thường được đặt sau các lớp tích chập (Conv2D) hoặc fully-connected (Dense).

a) ReLU

ReLU (Rectified Linear Unit) là hàm kích hoạt phổ biến nhất trong CNN nhờ đơn giản và hiệu quả:

$$\text{ReLU}(x) = \max(0, x)$$

Ưu điểm:

Tính toán nhanh, dễ huấn luyện.

Giảm hiện tượng *vanishing gradient* so với sigmoid/tanh.

Giúp mạng học nhanh và hội tụ tốt.

Trong đề tài này, ReLU được sử dụng trong các lớp Conv2D và Dense ẩn.

b) Sigmoid / Tanh

Sigmoid ánh xạ giá trị đầu vào về khoảng (0,1):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tanh ánh xạ về khoảng (-1,1):

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Nhận xét:

Sigmoid phù hợp trong bài toán nhị phân khi đầu ra cần dạng xác suất.

Tanh có trung bình gần 0 nên đôi khi hội tụ tốt hơn sigmoid.

Tuy nhiên cả hai đều dễ gây *vanishing gradient* khi $|x|$ lớn, làm huấn luyện mạng sâu khó khăn.

Trong các mạng CNN hiện đại, sigmoid/tanh ít dùng ở tầng ẩn, thay vào đó chủ yếu dùng ReLU.

c) Softmax

Softmax thường dùng ở *tầng đầu ra* của bài toán phân loại nhiều lớp, biến các logit thành phân phối xác suất.

Với vector đầu ra $z = [z_1, z_2, \dots, z_K]$, softmax được định nghĩa:

$$\text{Softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Ý nghĩa:

Trả về xác suất từng lớp.

Tổng xác suất bằng 1, phù hợp để lựa chọn lớp dự đoán.

Trong đề tài, mô hình phân loại 2 lớp sử dụng tầng cuối:

Dense(2, activation="softmax")

Nhờ đó mô hình xuất ra vector xác suất $[p_0, p_1]$ và chọn lớp có xác suất lớn nhất làm kết quả dự đoán.

3.4. Hàm mất mát (Loss Function)

Trong học sâu, *hàm mất mát (loss function)* dùng để đo mức độ sai lệch giữa *nhãn dự đoán* của mô hình và *nhãn thật* của dữ liệu. Mục tiêu của quá trình huấn luyện là điều chỉnh các tham số của mạng sao cho loss giảm dần và đạt giá trị nhỏ nhất.

a) Binary Cross Entropy / Categorical Cross Entropy

Binary Cross Entropy (BCE) thường dùng cho bài toán phân loại nhị phân khi tầng cuối có 1 neuron với sigmoid:

$$L_{BCE}(y, \hat{y}) = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Trong đó:

$y \in \{0, 1\}$ là nhãn thật,

$\hat{y} \in (0, 1)$ là xác suất dự đoán.

Categorical Cross Entropy (CCE) dùng cho phân loại nhiều lớp khi nhãn dạng one-hot:

$$L_{CCE}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

Trong đó:

K là số lớp,

$y = [y_1, \dots, y_K]$ là nhãn one-hot,

$y^{\wedge} = [y^{\wedge}_1, \dots, y^{\wedge}_K]$ là vector xác suất dự đoán (softmax output).

b) Ý nghĩa và vai trò trong huấn luyện

Hàm loss là “kim chỉ nam” giúp mô hình học đúng hướng:

Đo sai số dự đoán: loss càng nhỏ $\rightarrow$ dự đoán càng gần nhãn thật.

Cơ sở để cập nhật trọng số: thuật toán lan truyền ngược (backpropagation) tính gradient của loss để cập nhật tham số mạng:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Theo dõi quá trình học: quan sát loss của train và validation giúp đánh giá:

mô hình học tốt hay không,

có bị *overfitting* hay *underfitting*,

có cần thay đổi kiến trúc hoặc hyperparameter.

Tóm lại, lựa chọn loss phù hợp giúp mô hình tối ưu đúng mục tiêu phân loại và nâng cao chất lượng dự đoán.

3.5. Thuật toán tối ưu (Optimization)

Trong quá trình huấn luyện mạng nơ-ron, mục tiêu là tìm bộ tham số θ (trọng số và bias) sao cho hàm mất mát nhỏ nhất:

$$\theta^* = \arg \min_{\theta} \mathcal{J}(\theta)$$

Để thực hiện điều này, ta sử dụng các *thuật toán tối ưu* nhằm cập nhật tham số dựa trên gradient của hàm mất mát theo hướng giảm dần.

a) Gradient Descent

Gradient Descent (GD) là phương pháp tối ưu cơ bản nhất, cập nhật tham số theo gradient trên toàn bộ tập dữ liệu:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{J}(\theta)$$

Trong đó:

η là learning rate,

$\nabla_{\theta} \mathcal{J}(\theta)$ là gradient của loss theo tham số.

Ưu điểm:

Đơn giản, dễ hiểu.

Hạn chế:

Mỗi lần cập nhật cần duyệt toàn bộ dữ liệu $\rightarrow$ tốn thời gian với dataset lớn.

Dễ mắc kẹt ở điểm yên ngựa hoặc hội tụ chậm.

b) SGD + Momentum

Stochastic Gradient Descent (SGD) cải tiến GD bằng cách cập nhật tham số theo từng batch (mini-batch) thay vì toàn bộ dữ liệu:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{J}(\theta; \text{batch})$$

Nhờ đó SGD:

huấn luyện nhanh hơn,

cập nhật thường xuyên hơn,

nhưng gradient bị nhiễu dẫn đến dao động.

Để giảm dao động và giúp hội tụ nhanh hơn, ta dùng *Momentum*. Ý tưởng là thêm “quán tính” cho hướng cập nhật:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

Trong đó:

g_t là gradient tại bước ttt,

v_t là vận tốc (velocity),

β thường chọn khoảng 0.9.

Ưu điểm:

Tăng tốc hội tụ theo hướng đúng.

Giảm dao động trên bề mặt loss.

c) *Adam*

Adam (Adaptive Moment Estimation) là một trong những optimizer phổ biến nhất hiện nay, kết hợp ưu điểm của:

Momentum (moment bậc 1)

RMSProp (moment bậc 2)

Adam lưu hai thống kê:

moment bậc 1 (trung bình gradient)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

moment bậc 2 (trung bình bình phương gradient)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

Cập nhật tham số:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Ưu điểm:

Tự điều chỉnh learning rate theo từng tham số.

Hội tụ nhanh, ổn định.

Hiệu quả cho dữ liệu nhiễu và mô hình sâu (CNN).

3.6. Các kỹ thuật tránh overfitting

Overfitting xảy ra khi mô hình học quá kỹ dữ liệu huấn luyện (train) dẫn đến hiệu năng cao trên train nhưng giảm trên validation/test. Hiện tượng này thường gặp trong học sâu do mạng có nhiều tham số và khả năng “ghi nhớ” dữ liệu mạnh. Để giảm overfitting, nhiều kỹ thuật regularization được sử dụng trong quá trình huấn luyện.

a) Regularization (L2)

L2 regularization (weight decay) thêm một thành phần phạt vào hàm mất mát để hạn chế trọng số quá lớn:

$$\mathcal{J}'(\theta) = \mathcal{J}(\theta) + \lambda \sum_i \theta_i^2$$

Trong đó:

$\mathcal{J}(\theta)$ là loss gốc,

λ là hệ số regularization.

Ý nghĩa:

khuyến khích trọng số nhỏ,

giúp mô hình tổng quát tốt hơn,

giảm nguy cơ mô hình ghi nhớ dữ liệu.

b) Dropout

Dropout là kỹ thuật “tắt ngẫu nhiên” một số neuron trong quá trình huấn luyện với xác suất p . Điều này giúp mô hình không phụ thuộc vào một vài neuron cụ thể, từ đó giảm overfitting.

Nếu neuron được giữ lại với xác suất $q = 1-p$, ta có thể mô tả:

$$h' = h \cdot r$$

với r là mask ngẫu nhiên (0 hoặc 1).

Ưu điểm:

- tăng tính robust,
- giảm tương quan giữa các neuron,
- thường được dùng ở các lớp Dense.

c) Data Augmentation

Data Augmentation là kỹ thuật tăng số lượng mẫu huấn luyện bằng cách biến đổi ảnh theo nhiều cách nhưng vẫn giữ nhãn không đổi. Các phép biến đổi thường dùng:

- xoay ảnh (rotation),
- lật ảnh (flip),
- phóng to/thu nhỏ (zoom),
- dịch chuyển (shift),
- thay đổi độ sáng/độ tương phản nhẹ.

Lợi ích:

- giúp mô hình học đa dạng hơn,
- giảm phụ thuộc vào một kiểu ảnh,
- tăng khả năng tổng quát hóa trên dữ liệu mới.

Đối với ảnh X-quang, augmentation cần được thực hiện hợp lý để không làm mất ý nghĩa y khoa của ảnh.

d) *Early Stopping*

Early Stopping dừng huấn luyện sớm khi chất lượng mô hình trên tập validation không cải thiện sau một số epoch liên tiếp. Cụ thể, nếu:

validation loss tăng hoặc

validation accuracy không tăng

trong nhiều epoch, thì dừng training và giữ lại mô hình tốt nhất.

Ưu điểm:

tránh huấn luyện quá mức gây overfitting,

tiết kiệm tài nguyên tính toán,

thường kết hợp với ModelCheckpoint để lưu model tốt nhất.

3.7. *Chỉ số đánh giá (Evaluation Metrics)*

Trong bài toán phân loại ảnh, đặc biệt với dữ liệu y khoa, chỉ sử dụng Accuracy là chưa đủ vì dữ liệu có thể mất cân bằng và sai lệch giữa các lớp. Do đó, báo cáo sử dụng các chỉ số đánh giá phổ biến gồm Accuracy, Precision, Recall, F1-score và Confusion Matrix để đánh giá mô hình toàn diện hơn.

a) *Accuracy*

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Trong đó:

TP (True Positive): dự đoán đúng lớp dương

TN (True Negative): dự đoán đúng lớp âm

FP (False Positive): dự đoán nhầm âm thành dương

FN (False Negative): dự đoán nhầm dương thành âm

Ý nghĩa: Accuracy dễ hiểu và trực quan, tuy nhiên khi dữ liệu mất cân bằng, accuracy có thể gây hiểu nhầm về chất lượng thật sự của mô hình.

b) Precision, Recall

Precision đo mức độ chính xác trong các dự đoán dương:

$$Precision = \frac{TP}{TP + FP}$$

Recall (Sensitivity) đo khả năng mô hình tìm đúng các mẫu dương:

$$Recall = \frac{TP}{TP + FN}$$

Ý nghĩa:

Precision cao $\rightarrow$ ít dự đoán nhầm dương (FP thấp).

Recall cao $\rightarrow$ ít bỏ sót mẫu dương (FN thấp).

Trong ảnh y khoa, Recall thường quan trọng vì bỏ sót trường hợp bệnh có thể gây hậu quả nghiêm trọng.

c) F1-score

F1-score là trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Ý nghĩa:

F1-score cân bằng giữa Precision và Recall, đặc biệt hữu ích khi dữ liệu không cân bằng hoặc khi cần đánh giá tổng hợp chất lượng phân loại.

d) Confusion Matrix

Confusion Matrix (ma trận nhầm lẫn) thể hiện chi tiết số lượng dự đoán đúng/sai theo từng lớp.

Với bài toán 2 lớp, confusion matrix có dạng:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Ý nghĩa:

Cho thấy mô hình nhầm lẫn giữa các lớp ra sao.

Giúp phân tích lỗi trực quan: mô hình thường nhầm lớp nào nhiều hơn.

Là cơ sở để tính các chỉ số Precision, Recall, F1.

Trong báo cáo này, Confusion Matrix và Classification Report được dùng để đánh giá chi tiết hiệu năng mô hình trên tập kiểm tra (test).

IV. Nghiên cứu liên quan

4.1. Các phương pháp truyền thống

Trước khi học sâu phát triển mạnh, các bài toán phân loại ảnh (bao gồm ảnh y khoa) thường được giải theo hướng: *trích xuất đặc trưng thủ công* → *đưa vào bộ phân loại học máy*. Hiệu quả của các phương pháp này phụ thuộc rất lớn vào chất lượng đặc trưng được thiết kế.

a) *Trích xuất đặc trưng thủ công (HOG, LBP, ...)*

Đặc trưng thủ công (handcrafted features) là các đặc trưng được thiết kế dựa trên kiến thức chuyên môn, nhằm mô tả các thuộc tính hình dạng, kết cấu hoặc biên cạnh của ảnh.

Một số phương pháp phổ biến:

HOG (Histogram of Oriented Gradients): trích xuất thông tin hướng gradient, mô tả biên và hình dạng vật thể.

LBP (Local Binary Patterns): mô tả kết cấu (texture) bằng cách mã hóa mối quan hệ cục bộ giữa các điểm ảnh.

SIFT/SURF (ít dùng hơn hiện nay): trích xuất keypoint và mô tả đặc trưng cục bộ.

Gabor filter / Wavelet: trích xuất kết cấu và tần số không gian.

Trong ảnh X-quang, các phương pháp này có thể giúp mô tả độ mờ/kết cấu ở vùng phổi, tuy nhiên thường khó bao quát hết đặc trưng bệnh lý đa dạng.

b) Phân loại bằng ML cổ điển (SVM, KNN, ...)

Sau khi trích xuất đặc trưng, ảnh được biểu diễn thành vector và đưa vào mô hình học máy cổ điển để phân loại.

Một số thuật toán phổ biến:

SVM (Support Vector Machine): mạnh trong dữ liệu có biên phân tách rõ, có thể dùng kernel để phân tách phi tuyến.

KNN (K-Nearest Neighbors): dự đoán dựa trên “láng giềng gần nhất”, đơn giản nhưng phụ thuộc mạnh vào khoảng cách và dữ liệu.

Logistic Regression: mô hình tuyến tính cho phân loại, dễ triển khai.

Random Forest / Decision Tree: học theo cây, trực quan nhưng dễ overfitting nếu dữ liệu nhiều.

Hạn chế chung của hướng tiếp cận truyền thống:

cần thiết kế đặc trưng thủ công → phụ thuộc kinh nghiệm,

khó mở rộng cho dữ liệu phức tạp,

hiệu quả giảm khi dữ liệu đa dạng hoặc nhiều.

4.2. Các phương pháp học sâu hiện đại

Sự phát triển của Deep Learning, đặc biệt là CNN, đã thay đổi hoàn toàn hướng tiếp cận trong phân loại ảnh. Thay vì thiết kế đặc trưng thủ công, mô hình học sâu *tự học đặc trưng trực tiếp từ dữ liệu*.

a) CNN end-to-end

CNN end-to-end là mô hình học sâu được huấn luyện trực tiếp từ ảnh thô đến nhãn dự đoán, bao gồm:

trích xuất đặc trưng (Conv + Pooling),

học biểu diễn (Flatten/Dense),

phân loại (Softmax/Sigmoid).

Ưu điểm:

- tự động học đặc trưng phù hợp dữ liệu,
- hiệu quả cao với ảnh,
- đễ mở rộng bằng tăng dữ liệu hoặc tăng độ sâu mạng.

Trong đề tài , mô hình *CNN xây dựng từ đầu bằng Keras* thuộc nhóm phương pháp CNN end-to-end.

b) Transfer Learning

Transfer Learning (học chuyển giao) là phương pháp sử dụng mô hình đã được huấn luyện trước trên bộ dữ liệu lớn (ví dụ ImageNet), sau đó:

- dùng như bộ trích xuất đặc trưng,
- hoặc fine-tuning để phù hợp với dữ liệu mới.

Ưu điểm nổi bật:

- tăng độ chính xác khi dữ liệu ít,
- giảm thời gian huấn luyện,
- mô hình học được đặc trưng mạnh hơn.

Trong ảnh y khoa, transfer learning rất phổ biến vì dữ liệu thường hạn chế và khó thu thập.

4.3. Các backbone phổ biến trong thị giác máy tính

Backbone là phần mạng chính dùng để trích xuất đặc trưng. Một số backbone nổi bật được sử dụng rộng rãi:

a) VGG

- Kiến trúc đơn giản: nhiều lớp Conv 3×3 nối tiếp.
- Dễ hiểu, dễ triển khai.
- Nhược điểm: số tham số lớn $\rightarrow$ tốn tài nguyên.

b) ResNet

Giới thiệu *residual connection* giúp mạng rất sâu vẫn huấn luyện tốt.

Hiệu quả cao và ổn định trong thực nghiệm.

Thường được dùng trong transfer learning.

c) MobileNet

Thiết kế nhẹ, tối ưu cho thiết bị di động.

Dùng *Depthwise Separable Convolution* giảm tham số đáng kể.

Phù hợp triển khai thực tế (deployment).

d) EfficientNet / DenseNet

DenseNet: kết nối dày đặc giúp tái sử dụng đặc trưng, giảm gradient vanishing.

EfficientNet: tối ưu cân bằng độ sâu – độ rộng – độ phân giải; đạt hiệu năng cao với chi phí hợp lý.

Các backbone này thường cho kết quả rất tốt trong các bài toán phân loại ảnh và đặc biệt hữu ích trong dữ liệu y khoa.

4.4. Nhận xét – hạn chế – định hướng tiếp cận của đề tài

Từ tổng quan các phương pháp ở trên, có thể rút ra một số nhận xét:

Các phương pháp truyền thống (HOG/LBP + SVM/KNN) có ưu điểm đơn giản và dễ triển khai, tuy nhiên:

- + phụ thuộc mạnh vào đặc trưng thủ công,
- + hiệu quả hạn chế trên dữ liệu ảnh phức tạp như X-quang,
- + khó tổng quát tốt khi dữ liệu đa dạng.

Các phương pháp học sâu (CNN) có ưu điểm vượt trội trong ảnh nhờ:

- + tự động học đặc trưng,
- + mô hình hóa tốt các mẫu phức tạp,
- + hiệu năng cao khi dữ liệu đủ.

Tuy nhiên, học sâu cũng tồn tại các hạn chế:

- cần tài nguyên tính toán (GPU),
- dễ overfitting nếu dữ liệu nhỏ,
- khó giải thích (black-box).

Định hướng tiếp cận của đề tài:

Trong phạm vi môn học, báo cáo lựa chọn hướng tiếp cận theo mô hình *CNN end-to-end xây dựng từ đầu* bằng TensorFlow/Keras. Lý do:

- phù hợp với kiến thức môn Học sâu,
- dễ triển khai và giải thích cấu trúc mạng,
- kiểm soát được pipeline huấn luyện – đánh giá.

Đồng thời, báo cáo cũng đặt nền tảng để mở rộng trong tương lai theo hướng:

- sử dụng transfer learning với backbone (ResNet/MobileNet/EfficientNet),
- fine-tuning để tăng độ chính xác,
- trực quan hóa vùng chú ý (Grad-CAM) để nâng tính giải thích.

V. Dữ liệu và tiền xử lý

5.1. Giới thiệu bộ dữ liệu sử dụng

Dữ liệu là yếu tố cốt lõi quyết định hiệu quả của mô hình học sâu. Trong đề tài này, nhóm sử dụng bộ dữ liệu ảnh *X-quang ngực (Chest X-ray)* đã được gán nhãn, phục vụ bài toán phân loại ảnh nhị phân. Dữ liệu được tổ chức theo cấu trúc thư mục rõ ràng và được đọc trực tiếp bằng thư viện TensorFlow/Keras thông qua hàm `image_dataset_from_directory`.

a) Nguồn dữ liệu

Bộ dữ liệu được cung cấp dưới dạng file nén và được lưu trữ trên Google Drive của người thực hiện. Trong notebook, dữ liệu được:

Mount Google Drive vào môi trường Google Colab

Giải nén file X-ray.zip

Tạo cấu trúc thư mục để huấn luyện theo dạng:

train/ : tập huấn luyện

val/ : tập kiểm định (validation)

test/ : tập kiểm tra (testing)

Việc tổ chức dữ liệu thành ba tập riêng biệt giúp:

mô hình học trên train,

điều chỉnh siêu tham số dựa trên validation,

đánh giá khách quan bằng test.

b) Mô tả các lớp (Normal / Pneumonia hoặc tương ứng)

Đây là bài toán *phân loại 2 lớp*, tức mỗi ảnh X-quang sẽ thuộc về một trong hai nhãn:

Lớp 0: Normal (Bình thường)

Ảnh X-quang của người có phổi bình thường, không có dấu hiệu viêm nhiễm rõ rệt.

Lớp 1: Pneumonia (Viêm phổi) / lớp bệnh tương ứng trong dữ liệu

Ảnh có dấu hiệu bất thường, thường thể hiện dưới dạng vùng mờ, đông đặc hoặc thay đổi kết cấu tại vùng phổi.

Trong quá trình đọc dữ liệu, TensorFlow tự động:

lấy tên lớp theo tên folder con,

gán nhãn tương ứng dạng số (0/1),

tạo batch dữ liệu để đưa vào mô hình CNN.

5.2. Thống kê dữ liệu và phân bố lớp

Sau khi giải nén và tổ chức dữ liệu theo cấu trúc thư mục, nhóm tiến hành thống kê số lượng ảnh trong từng tập dữ liệu và phân bố giữa các lớp. Việc thống kê này rất quan trọng nhằm:

đánh giá quy mô dữ liệu,

kiểm tra sự cân bằng giữa các lớp (class imbalance),

dự đoán nguy cơ mô hình bị thiên lệch khi huấn luyện

Trong bài toán phân loại nhị phân, dữ liệu được chia theo hai lớp *Normal* và *Pneumonia* (hoặc lớp *tương ứng*). Tổng số lượng ảnh trong từng lớp được tính riêng cho 3 tập: Train, Validation và Test.

Bảng dưới đây thể hiện số lượng ảnh ở từng tập dữ liệu và từng lớp:

Bảng 5.1. Thống kê số lượng ảnh và phân bố lớp

Tập dữ liệu	Normal	Pneumonia	Tổng
Train	1341	3875	5216
Validation	8	8	1043
Test	234	390	624

Nếu phân bố ảnh giữa hai lớp chênh lệch lớn, mô hình có thể dễ thiên về lớp chiếm đa số. Khi đó cần quan tâm tới các chỉ số đánh giá như *Precision*, *Recall*, *F1-score* thay vì chỉ dựa trên *Accuracy*. Ngoài ra có thể áp dụng một số giải pháp như:

tăng cường dữ liệu (augmentation),

sử dụng *class weight* khi huấn luyện.

5.3. Chia tập Train / Validation / Test

Để đảm bảo quá trình huấn luyện và đánh giá mô hình khách quan, dữ liệu được chia thành ba tập riêng biệt:

Train set: dùng để huấn luyện mô hình, cập nhật tham số qua từng epoch.

Validation set: dùng để theo dõi chất lượng mô hình trong quá trình huấn luyện và hỗ trợ điều chỉnh siêu tham số (hyperparameters).

Test set: dùng để đánh giá cuối cùng sau khi mô hình đã huấn luyện xong, phản ánh khả năng tổng quát hóa trên dữ liệu chưa từng thấy.

Trong notebook, dữ liệu đã được tổ chức sẵn theo cấu trúc thư mục:

```
train/
```

```
val/
```

```
test/
```

Sau đó, nhóm sử dụng hàm:

```
tf.keras.utils.image_dataset_from_directory(...)
```

để tự động:

lấy ảnh theo từng thư mục lớp,

gán nhãn tương ứng,

tạo dữ liệu theo dạng batch phục vụ huấn luyện.

Việc chia tập theo cách này giúp tránh rò rỉ dữ liệu (data leakage), đảm bảo tính công bằng và chính xác khi đánh giá hiệu năng mô hình.

5.4. *Tiền xử lý dữ liệu*

Trong bài toán phân loại ảnh, bước tiền xử lý đóng vai trò quan trọng nhằm đưa dữ liệu về dạng chuẩn hóa, giúp mô hình huấn luyện ổn định hơn, hội tụ nhanh hơn và cải thiện khả năng tổng quát hóa. Đối với ảnh X-quang, tiền xử lý còn giúp giảm ảnh hưởng của sự khác biệt chất lượng ảnh và chuẩn hóa dữ liệu đầu vào cho CNN.

a) *Chuẩn hóa kích thước ảnh (resize)*

Các ảnh trong dataset thường có kích thước không đồng nhất. Trong khi đó, CNN yêu cầu đầu vào phải có kích thước cố định. Vì vậy, nhóm tiến hành chuẩn hóa toàn bộ ảnh về cùng kích thước:

180×180

Việc resize được thực hiện trực tiếp khi tạo dataset bằng TensorFlow:

```
image_dataset_from_directory(..., image_size=(180, 180))
```

Ý nghĩa của bước resize:

đảm bảo dữ liệu đầu vào đồng nhất,

phù hợp với kiến trúc mạng,

giảm chi phí tính toán so với ảnh có độ phân giải quá lớn.

b) Chuẩn hóa giá trị pixel (normalize/rescale)

Giá trị pixel ảnh ban đầu thuộc miền $[0, 255]$, nếu đưa trực tiếp vào mạng sẽ khiến gradient dao động mạnh và làm mô hình khó hội tụ. Do đó, nhóm chuẩn hóa pixel về miền $[0, 1]$ bằng phép chia 255:

$$X_{norm} = \frac{X}{255}$$

Trong notebook, bước này được triển khai bằng lớp:

```
layers.Rescaling(1./255)
```

Lợi ích:

giúp mô hình huấn luyện ổn định hơn,
tăng tốc hội tụ,
giảm nguy cơ gradient exploding/vanishing.

c) Xử lý kênh ảnh (RGB/Grayscale)

Ảnh X-quang về bản chất thường là ảnh mức xám (grayscale). Tuy nhiên, khi đọc dữ liệu bằng `image_dataset_from_directory`, TensorFlow mặc định tải ảnh ở dạng **RGB (3 kênh)**, tức:

$$X \in \mathbb{R}^{180 \times 180 \times 3}$$

Việc giữ ảnh ở dạng RGB trong đề tài giúp:

thuận tiện cho pipeline đọc dữ liệu mặc định,
tương thích với các kiến trúc CNN phổ biến (đặc biệt khi mở rộng sang Transfer Learning với ImageNet).

Ngoài ra, trong một số hướng phát triển, có thể chuyển ảnh sang *Grayscale (1 kênh)* để:

giảm số tham số,
giảm tài nguyên tính toán,
nhưng vẫn giữ thông tin cần thiết vì X-quang chủ yếu dựa trên cường độ sáng/tối.

5.5. Tăng cường dữ liệu (*Data Augmentation*)

Trong học sâu, hiệu năng mô hình phụ thuộc mạnh vào quy mô và độ đa dạng của dữ liệu huấn luyện. Tuy nhiên, trong các bài toán ảnh y khoa như X-quang, dữ liệu thường không quá lớn và khó thu thập. Vì vậy, kỹ thuật *tăng cường dữ liệu (Data Augmentation)* được áp dụng nhằm tạo ra thêm các biến thể của ảnh huấn luyện mà vẫn giữ nguyên nhãn, từ đó giảm overfitting và cải thiện khả năng tổng quát hóa của mô hình.

a) *Flip / Rotation / Zoom / Shift*

Một số phép tăng cường dữ liệu phổ biến thường được sử dụng gồm:

Flip (lật ảnh):

Lật ngang (horizontal flip) giúp mô hình không phụ thuộc vào hướng trái/phải của ảnh.

Lưu ý: Với ảnh y khoa, việc flip cần cân nhắc vì có thể đảo trái – phải cơ quan, tuy nhiên với bài toán phân loại nhị phân thường vẫn có thể sử dụng ở mức hợp lý.

Rotation (xoay ảnh):

Xoay ảnh với góc nhỏ (ví dụ $\pm 10 - 15$ độ) nhằm tạo ra các ảnh ở nhiều góc chụp khác nhau.

Giúp mô hình giảm phụ thuộc vào tư thế chụp cụ thể.

Zoom (phóng to/thu nhỏ):

Mô phỏng sự khác nhau về khoảng cách chụp hoặc vùng phổi hiển thị nhiều/ít.

Tăng tính linh hoạt khi mô hình gặp ảnh có tỷ lệ khác nhau.

Shift (dịch chuyển):

Dịch ảnh theo chiều ngang/dọc một đoạn nhỏ.

Giúp mô hình tập trung học đặc trưng tổng quát thay vì phụ thuộc vị trí cố định.

Các phép biến đổi trên thường được áp dụng ngẫu nhiên trong quá trình huấn luyện để mỗi epoch mô hình được nhìn thấy nhiều “phiên bản” khác nhau của cùng một ảnh.

b) Ý nghĩa và tác dụng

Data Augmentation mang lại nhiều lợi ích quan trọng:

Tăng độ đa dạng của tập huấn luyện mà không cần thu thập thêm dữ liệu mới.

Giảm overfitting, giúp mô hình không học thuộc (memorize) các ảnh train.

Cải thiện khả năng tổng quát hóa, giúp mô hình dự đoán tốt hơn trên ảnh mới trong tập test.

Tăng độ bền vững (robustness) trước các biến đổi nhỏ trong dữ liệu thực tế như ảnh lệch góc, sáng tối khác nhau hoặc bố cục thay đổi.

Đối với ảnh X-quang, augmentation còn giúp mô hình thích ứng tốt hơn với sự khác biệt trong điều kiện chụp và chất lượng ảnh giữa các nguồn khác nhau.

VI. Phương pháp đề xuất

6.1. Ý tưởng tổng quát và kiến trúc hệ thống

Trong đề tài này, nhóm đề xuất một phương pháp tiếp cận dựa trên *mạng nơ-ron tích chập (CNN)* nhằm giải quyết bài toán phân loại ảnh X-quang ngực theo hai lớp. Ý tưởng chính là tận dụng khả năng của CNN trong việc *tự động trích xuất đặc trưng không gian* từ ảnh (ví dụ: vùng mờ, vùng đông đặc, kết cấu bất thường trong phổi), từ đó thực hiện phân loại chính xác mà không cần thiết kế đặc trưng thủ công.

Thay vì sử dụng các phương pháp truyền thống gồm bước trích xuất đặc trưng bằng tay (HOG, LBP, ...) và phân loại bằng thuật toán ML, phương pháp học sâu trong đề tài thực hiện theo hướng *end-to-end*, tức mô hình học trực tiếp từ dữ liệu ảnh đầu vào đến nhãn đầu ra.

a) Ý tưởng tổng quát

Phương pháp được xây dựng theo pipeline gồm 4 giai đoạn chính:

Bước 1: Chuẩn bị dữ liệu

đọc ảnh từ các thư mục train/val/test,

gán nhãn tự động theo tên thư mục lớp.

Bước 2: Tiền xử lý

chuẩn hóa kích thước ảnh về 180×180 ,

chuẩn hóa pixel về miền $[0,1]$ bằng Rescaling ($1./255$).

Bước 3: Huấn luyện mô hình CNN

sử dụng nhiều lớp Conv2D + MaxPooling để trích xuất đặc trưng ảnh,
các lớp Dense để thực hiện phân loại.

Bước 4: Đánh giá và suy luận

đánh giá mô hình trên tập test bằng Accuracy, Confusion Matrix, Classification Report,

thực hiện dự đoán nhãn cho ảnh mới.

b) Kiến trúc hệ thống đề xuất

Hệ thống tổng quát có thể mô tả theo sơ đồ như sau:

Ảnh X-quang đầu vào

→ (Resize + Normalize)

→ CNN Feature Extractor (Conv + Pooling)

→ Classifier (Dense + Softmax)

→ Dự đoán nhãn lớp

Cụ thể:

Khởi tiền xử lý (Preprocessing Block)

Resize ảnh về 180×180 .

Rescaling pixel: $X \leftarrow X/255$

Khởi trích xuất đặc trưng (Feature Extraction Block)

Các lớp Conv2D kết hợp ReLU giúp học đặc trưng theo từng mức:

+ tầng đầu học biên/texture,

+ tầng sâu học cấu trúc và vùng bất thường phức tạp hơn.

MaxPooling giúp giảm kích thước feature map và chống nhiễu.

Khởi phân loại (Classification Block)

Flatten chuyển tensor đặc trưng thành vector 1 chiều.

Dense(128) học biểu diễn tổng quát.

Dense(2, Softmax) cho ra xác suất thuộc 2 lớp.

c) Ưu điểm của kiến trúc

Kiến trúc đề xuất có các ưu điểm chính:

Đơn giản, dễ triển khai bằng TensorFlow/Keras, phù hợp với phạm vi môn học.

Huấn luyện end-to-end, không cần đặc trưng thủ công.

Tốc độ huấn luyện tốt, phù hợp môi trường Colab.

Có khả năng mở rộng về sau theo hướng:

tăng số lớp CNN,

thêm regularization (Dropout/L2),

hoặc thay CNN bằng Transfer Learning.

6.2. Pipeline huấn luyện bằng thư viện học sâu (TensorFlow/Keras/PyTorch)

Trong đề tài này, mô hình được triển khai và huấn luyện bằng thư viện *TensorFlow/Keras* trên môi trường Google Colab. Toàn bộ pipeline được xây dựng theo các bước tuần tự gồm: chuẩn bị dữ liệu → xây dựng mô hình → compile → training → đánh giá và suy luận.

a) Chuẩn bị dữ liệu đầu vào

Dữ liệu ảnh X-quang được tổ chức theo cấu trúc thư mục train/val/test, trong đó mỗi thư mục con tương ứng với một lớp nhãn. Nhóm sử dụng hàm:

```
tf.keras.utils.image_dataset_from_directory()
```

để thực hiện các công việc:

đọc ảnh từ thư mục,

resize ảnh về 180×180 ,

tự động gán nhãn theo tên thư mục lớp,

chia dữ liệu theo batch.

Ngoài ra, để tối ưu tốc độ đọc dữ liệu khi huấn luyện, nhóm sử dụng pipeline theo hướng tăng hiệu năng:

batching (mini-batch),

prefetch (nạp trước dữ liệu vào RAM/GPU).

Nhờ đó, dữ liệu đầu vào của mô hình có dạng tensor:

$$X \in \mathbb{R}^{B \times 180 \times 180 \times 3}$$

với B là batch size.

b) Xây dựng model

Mô hình được xây dựng theo dạng *Sequential CNN*, bao gồm các thành phần:

Rescaling layer chuẩn hóa pixel về [0,1]

3 khối *Conv2D* + *MaxPooling2D* để trích xuất đặc trưng

Flatten + *Dense* để phân loại

Kiến trúc tổng quát:

Input: (180, 180, 3)

Rescaling(1/255)

Conv2D(16) + MaxPooling

Conv2D(32) + MaxPooling

Conv2D(64) + MaxPooling

Flatten

Dense(128, ReLU)

Dense(2, Softmax)

Trong đó:

Conv2D giúp học đặc trưng cục bộ (texture, vùng mờ...)

MaxPooling giúp giảm kích thước và giảm nhiễu

Softmax cho ra xác suất thuộc 2 lớp

c) *Compile*

Sau khi xây dựng mô hình, nhóm tiến hành compile để thiết lập các thành phần huấn luyện:

Optimizer: Adam

Loss function: Cross Entropy (phù hợp phân loại 2 lớp với Softmax)

Metric: Accuracy

Trong Keras, bước compile tương ứng với việc xác định:

$$\text{Model} = \{\text{Loss, Optimizer, Metrics}\}$$

Optimizer Adam giúp mô hình hội tụ nhanh, ổn định và phù hợp với CNN.

d) *Training*

Mô hình được huấn luyện theo mini-batch trên tập train và theo dõi hiệu năng trên validation. Quá trình training diễn ra trong *10 epochs*, với mục tiêu giảm loss và tăng độ chính xác dự đoán.

Trong mỗi epoch:

dữ liệu train được đưa qua mạng (forward propagation),

tính loss,

lan truyền ngược (backpropagation) để cập nhật trọng số.

Quá trình huấn luyện tạo ra các giá trị:

train_loss, train_accuracy

val_loss, val_accuracy

Các chỉ số này giúp theo dõi:

tốc độ hội tụ,

nguy cơ overfitting (train tăng nhưng val giảm).

e) *Evaluate + Inference*

Sau khi huấn luyện, mô hình được đánh giá trên tập test nhằm đo khả năng tổng quát hóa. Các chỉ số đánh giá gồm:

Accuracy

Confusion Matrix

Classification Report (Precision, Recall, F1-score)

Trong notebook, kết quả đạt được:

Test Accuracy $\approx 85\%$

Ngoài ra, hệ thống cũng hỗ trợ chế độ *Inference*, tức dự đoán nhãn cho ảnh mới bằng quy trình:

đọc ảnh $\rightarrow$ resize $\rightarrow$ rescale

đưa vào model predict

lấy nhãn theo xác suất lớn nhất:

$$\hat{y} = \arg \max(\text{Softmax output})$$

Kết quả inference gồm:

nhãn dự đoán,

xác suất tương ứng của từng lớp.

6.3. *Mô hình cơ sở xây dựng từ đầu (Baseline CNN)*

Trong đề tài này, nhóm xây dựng một mô hình CNN cơ sở (baseline) theo hướng đơn giản nhưng hiệu quả nhằm giải quyết bài toán phân loại ảnh X-quang ngực nhị phân. Mô hình được xây dựng từ đầu bằng TensorFlow/Keras, sử dụng kiến trúc tuần tự (Sequential) gồm các lớp tích chập kết hợp pooling để tự động trích xuất đặc trưng ảnh, sau đó phân loại bằng các lớp fully-connected.

a) Kiến trúc CNN

Kiến trúc mô hình baseline có thể mô tả tóm tắt như sau:

Input: $180 \times 180 \times 3$

Rescaling: chuẩn hóa pixel về $[0,1]$

Conv2D (16) + ReLU $\rightarrow$ MaxPooling2D

Conv2D (32) + ReLU $\rightarrow$ MaxPooling2D

Conv2D (64) + ReLU $\rightarrow$ MaxPooling2D

Flatten

Dense (128) + ReLU

Dense (2) + Softmax

Cấu hình số lượng filter tăng dần $16 \rightarrow 32 \rightarrow 64$ giúp mô hình học đặc trưng theo mức độ ngày càng trừu tượng và giàu thông tin hơn.

b) Mô tả các tầng

(1) Input layer

Ảnh đầu vào được chuẩn hóa về kích thước:

$$X \in \mathbb{R}^{180 \times 180 \times 3}$$

Trong đó 3 kênh màu là RGB theo cách load dữ liệu mặc định của TensorFlow.

(2) Rescaling layer

Sử dụng lớp:

`Rescaling(1./255)`

để đưa giá trị pixel từ $[0,255]$ về $[0,1]$, giúp mô hình hội tụ nhanh và ổn định hơn.

(3) Các lớp Conv2D + MaxPooling2D

Mô hình có 3 khối tích chập:

Khối 1: Conv2D(16, 3×3) + MaxPooling
 → học các đặc trưng đơn giản như biên, vùng sáng/tối.

Khối 2: Conv2D(32, 3×3) + MaxPooling
 → học đặc trưng mức trung gian như texture, vùng kết cấu.

Khối 3: Conv2D(64, 3×3) + MaxPooling
 → học đặc trưng mức cao hơn như cấu trúc bất thường và vùng tổn thương.

Ý nghĩa:

Conv2D trích xuất đặc trưng cục bộ,

MaxPooling giảm kích thước feature map, giữ lại đặc trưng nổi bật, giảm nhiễu.

(4) Flatten layer

Sau các khối Conv/Pooling, tensor feature map được chuyển thành vector 1 chiều:

$$v = \text{Flatten}(F)$$

để đưa vào lớp Dense.

(5) Dense layer

Dense(128, activation="relu")

Lớp này giúp mô hình học mối quan hệ tổng quát giữa các đặc trưng trích xuất được trước khi đi đến quyết định phân loại.

(6) Output layer

Dense(2, activation="softmax")

Lớp đầu ra trả về xác suất thuộc hai lớp:

$$\hat{\mathbf{y}} = [p_0, p_1]$$

và nhãn dự đoán:

$$\hat{y} = \arg \max(\hat{y})$$

c) Ưu / nhược điểm

Ưu điểm

Kiến trúc đơn giản, dễ triển khai và giải thích.

Huấn luyện nhanh trên Google Colab.

Phù hợp làm baseline cho bài toán phân loại ảnh X-quang.

Có thể mở rộng dễ dàng bằng cách thêm augmentation, dropout hoặc tăng số lớp Conv.

Nhược điểm

CNN xây từ đầu thường yêu cầu dữ liệu đủ lớn để đạt hiệu quả cao.

Chưa tận dụng được tri thức từ mô hình pretrained (transfer learning).

Nguy cơ overfitting nếu dữ liệu hạn chế hoặc nhiễu mạnh.

Khả năng tổng quát hóa có thể thấp hơn các mô hình backbone mạnh như ResNet/EfficientNet.

6.4. Mô hình học chuyển giao (Transfer Learning)

Trong bài toán phân loại ảnh y khoa, dữ liệu thường không quá lớn và có thể bị nhiễu. Do đó, mô hình CNN xây dựng từ đầu có thể gặp hạn chế về khả năng tổng quát hóa. Một hướng tiếp cận phổ biến nhằm cải thiện hiệu năng là *học chuyển giao (Transfer Learning)*, tức sử dụng một mô hình đã được huấn luyện trước trên tập dữ liệu lớn (thường là ImageNet) để trích xuất đặc trưng mạnh, sau đó huấn luyện lại tầng phân loại cho bài toán cụ thể.

Transfer Learning đặc biệt hiệu quả trong các bài toán:

dữ liệu vừa và nhỏ,

khó thu thập dữ liệu gán nhãn,

cần mô hình có độ chính xác cao nhanh chóng.

a) *Lựa chọn backbone (MobileNetV2 / ResNet / EfficientNet ...)*

Backbone là phần mạng dùng để trích xuất đặc trưng, thường là một CNN sâu được pretrained. Một số backbone phổ biến:

MobileNetV2

nhẹ, tốc độ nhanh, ít tham số,
phù hợp triển khai thực tế và hạn chế tài nguyên.

ResNet (ResNet50/101)

mạnh, ổn định, trích xuất đặc trưng tốt,
phù hợp cho bài toán ảnh phức tạp.

EfficientNet (B0-B7)

tối ưu cân bằng giữa độ sâu – độ rộng – độ phân giải,
hiệu năng cao với chi phí tính toán hợp lý.

Trong bối cảnh đề tài, *MobileNetV2* hoặc *EfficientNetB0* thường là lựa chọn phù hợp do:

dễ train trên Colab,
hiệu năng tốt trên ảnh y khoa.

b) *Tùy biến tầng phân loại (classification head)*

Mô hình pretrained chỉ đóng vai trò *trích xuất đặc trưng*, do đó cần thay thế tầng cuối để phù hợp bài toán 2 lớp.

Cấu trúc thường dùng:

Input (180×180×3)

Pretrained Backbone (include_top=False)

GlobalAveragePooling2D

Dense/Dropout (tùy chọn)

Dense(2, Softmax)

Trong đó:

`include_top=False` giúp bỏ phần classifier gốc của ImageNet.

`GlobalAveragePooling2D` thay `Flatten` để giảm tham số và tránh overfitting.

Đầu ra vẫn là vector xác suất:

$$\hat{\mathbf{y}} = [p_0, p_1]$$

c) Chiến lược đóng băng lớp (freeze) khi train lần 1

Ở giai đoạn đầu, backbone đã học được đặc trưng chung về ảnh (biên, texture, hình dạng). Vì vậy thường:

đóng băng (freeze) toàn bộ backbone

chỉ huấn luyện classification head

Mục tiêu:

tránh làm “hỏng” trọng số pretrained,

giảm thời gian huấn luyện,

mô hình ổn định và ít overfitting.

Trong Keras, freeze được thực hiện bằng:

```
backbone.trainable = False
```

d) Fine-tuning: mở một phần lớp (unfreeze) và huấn luyện tiếp

Sau khi huấn luyện head ổn định, mô hình có thể tiếp tục cải thiện bằng *fine-tuning*, tức:

mở (unfreeze) một số lớp trên cùng của backbone,

huấn luyện tiếp để backbone thích nghi với dữ liệu X-quang.

Chiến lược phổ biến:

chỉ unfreeze các block cuối (high-level features),

giữ nguyên các block đầu (low-level features).

Ví dụ:

MobileNetV2: unfreeze 20–30 layer cuối,

ResNet/EfficientNet: unfreeze 1–2 stage cuối.

e) Điều chỉnh siêu tham số khi fine-tuning (LR nhỏ, epochs, ...)

Fine-tuning có nguy cơ làm mô hình “quên” tri thức pretrained nếu learning rate quá lớn. Vì vậy cần thiết lập:

learning rate nhỏ hơn nhiều so với training head

ví dụ: 10^{-4} hoặc 10^{-5}

epochs nhỏ hơn

fine-tune 5–10 epochs (tùy dataset)

sử dụng callback:

EarlyStopping

ModelCheckpoint

ReduceLROnPlateau (scheduler)

Nhờ vậy fine-tuning giúp:

tăng accuracy,

tăng khả năng tổng quát hóa,

giảm overfitting so với CNN train từ đầu.

6.5. Chiến lược huấn luyện

Để mô hình CNN học hiệu quả và giảm nguy cơ overfitting, quá trình huấn luyện được thiết kế theo các thành phần: lựa chọn siêu tham số, lựa chọn optimizer/lr và các callback giám sát huấn luyện.

a) Hyperparameters (batch size, epochs)

Trong thực nghiệm, dữ liệu được đưa vào mô hình theo mini-batch nhằm tối ưu thời gian huấn luyện và ổn định gradient. Các siêu tham số quan trọng gồm:

Batch size: xác định số lượng ảnh được đưa vào mô hình trong mỗi bước cập nhật trọng số. Batch size ảnh hưởng tới tốc độ huấn luyện và độ nhiễu của gradient (batch nhỏ nhiễu hơn nhưng có thể tổng quát tốt hơn; batch lớn ổn định hơn nhưng tốn RAM/GPU).

Epochs: số lần mô hình duyệt qua toàn bộ tập train. Trong notebook, mô hình baseline được huấn luyện trong 10 epochs, đủ để quan sát xu hướng hội tụ và đánh giá hiệu năng ban đầu.

Ngoài ra, các tham số đi kèm như `image_size=(180, 180)` và `shuffle` khi đọc dữ liệu cũng góp phần cải thiện chất lượng huấn luyện.

b) Optimizer và learning rate

Mô hình được huấn luyện với *Adam optimizer* do khả năng hội tụ nhanh và ổn định trong các bài toán CNN. Adam kết hợp ưu điểm của Momentum và RMSProp, tự điều chỉnh learning rate theo từng tham số, giúp mô hình học hiệu quả ngay cả khi dữ liệu nhiễu.

Optimizer: Adam

Learning rate: sử dụng giá trị mặc định của Adam trong Keras (hoặc có thể tinh chỉnh theo thực nghiệm).

Trong trường hợp mở rộng sang *fine-tuning (transfer learning)*, learning rate nên giảm nhỏ (ví dụ 10^{-4} hoặc 10^{-5} để tránh làm thay đổi quá mạnh các trọng số pretrained).

c) EarlyStopping

EarlyStopping là kỹ thuật dừng huấn luyện sớm khi chất lượng trên tập validation không còn cải thiện, giúp:

hạn chế overfitting,

tiết kiệm thời gian và tài nguyên,

chọn được mô hình có khả năng tổng quát hóa tốt hơn.

Tiêu chí thường dùng:

theo dõi `val_loss` (ưu tiên) hoặc `val_accuracy`,

nếu không cải thiện sau `patience` `epoch` thì dừng.

d) ModelCheckpoint (lưu model tốt nhất)

Trong quá trình huấn luyện, mô hình có thể đạt kết quả tốt nhất ở một epoch nào đó rồi sau đó bị giảm do overfitting. Vì vậy, *ModelCheckpoint* được sử dụng để:

tự động lưu mô hình tốt nhất theo tiêu chí (thường là `val_loss` nhỏ nhất),

đảm bảo khi đánh giá test sẽ dùng đúng model tốt nhất.

ModelCheckpoint thường kết hợp với *EarlyStopping* để vừa dừng sớm vừa lưu được mô hình tối ưu.

6.6. Minh họa kiến trúc mô hình

Để trình bày rõ cấu trúc và quy trình hoạt động của hệ thống, báo cáo sử dụng hai dạng minh họa: sơ đồ pipeline và sơ đồ kiến trúc mạng.

a) Sơ đồ pipeline

Pipeline tổng quát của hệ thống phân loại ảnh X-quang:

Bước 1: Nhập ảnh X-quang từ `train/val/test`

Bước 2: Resize về 180×180

Bước 3: Rescale pixel về $[0,1]$

Bước 4: CNN trích xuất đặc trưng (Conv + Pooling)

Bước 5: Dense phân loại (Softmax)

Bước 6: Xuất nhãn dự đoán + xác suất

Có thể biểu diễn ngắn gọn:

Input Image $\rightarrow$ ***Preprocess*** $\rightarrow$ ***CNN Feature Extractor*** $\rightarrow$ ***Classifier*** $\rightarrow$ ***Output***

b) Sơ đồ mạng (plot model)

Mô hình baseline CNN trong đề tài có kiến trúc:

Input($180 \times 180 \times 3$)

Rescaling(1/255)

Conv2D(16) + MaxPooling

Conv2D(32) + MaxPooling

Conv2D(64) + MaxPooling

Flatten

Dense(128, ReLU)

Dense(2, Softmax)

Trong báo cáo, sơ đồ mạng có thể được minh họa bằng:

bảng tóm tắt `model.summary()` (tên layer + output shape + số tham số)

hình vẽ kiến trúc từ `plot_model(model, show_shapes=True)`

Các minh họa này giúp người đọc nắm rõ:

luồng dữ liệu qua các tầng,

kích thước tensor biến đổi qua từng bước,

tổng số tham số của mô hình.

VII. Thiết kế thực nghiệm (Experimental Setup)

7.1. Môi trường thực nghiệm

Đề tài được triển khai và huấn luyện mô hình trên môi trường lập trình Python kết hợp thư viện học sâu TensorFlow/Keras. Việc sử dụng môi trường có hỗ trợ GPU giúp tăng tốc quá trình huấn luyện mô hình CNN trên dữ liệu ảnh.

a) Nền tảng (Google Colab/local)

Nhóm thực hiện toàn bộ quá trình thực nghiệm trên *Google Colab*, bao gồm:

mount Google Drive để truy cập dữ liệu và lưu mô hình,

giải nén bộ dữ liệu từ file .zip,

huấn luyện và đánh giá mô hình trực tiếp trên notebook.

Ưu điểm của Colab:

miễn phí và dễ sử dụng,
hỗ trợ GPU,
thuận tiện chia sẻ và tái sử dụng notebook.

b) GPU/CPU/RAM

Trong quá trình huấn luyện, nhóm sử dụng tài nguyên tính toán do Colab cung cấp, gồm:

CPU để xử lý dữ liệu,
GPU để tăng tốc huấn luyện CNN,
RAM phục vụ lưu batch dữ liệu và mô hình.

Việc huấn luyện trên GPU giúp giảm đáng kể thời gian training so với chỉ dùng CPU, đặc biệt khi huấn luyện nhiều epoch trên tập train lớn.

c) Version thư viện

Các thư viện chính được sử dụng trong thực nghiệm:

Python 3.x
TensorFlow / Keras
NumPy, Matplotlib (phục vụ xử lý và trực quan hóa)
scikit-learn (Confusion Matrix, Classification Report)

7.2. Thiết lập huấn luyện

Các tham số huấn luyện (hyperparameters) được thiết lập dựa trên đặc thù dữ liệu và khả năng tính toán của môi trường Colab.

a) Input size

Ảnh đầu vào được chuẩn hóa về kích thước:

$$180 \times 180 \times 3$$

Việc chuẩn hóa kích thước giúp dữ liệu đồng nhất và phù hợp với mô hình CNN.

b) Batch size

Dữ liệu được đưa vào mô hình theo mini-batch. Batch size được lựa chọn nhằm cân bằng giữa:

tốc độ training,

khả năng tận dụng GPU,

giới hạn RAM/GPU.

Batch size được thiết lập trực tiếp khi tạo dataset bằng:

```
image_dataset_from_directory(..., batch_size=...)
```

c) Epochs

Mô hình baseline CNN được huấn luyện trong:

10 epochs

Epochs đủ để mô hình học được các đặc trưng cơ bản và quan sát được xu hướng hội tụ thông qua các chỉ số loss/accuracy trên train và validation.

d) Learning rate

Mô hình sử dụng *Adam optimizer*, learning rate mặc định (hoặc được tinh chỉnh nếu cần). Learning rate quyết định tốc độ cập nhật tham số:

LR quá lớn → dễ dao động, khó hội tụ

LR quá nhỏ → học chậm

Trong trường hợp mở rộng sang fine-tuning (transfer learning), learning rate được khuyến nghị giảm nhỏ (ví dụ 10^{-4} hoặc 10^{-5} để giữ ổn định trọng số pretrained).

7.3. Baseline và mô hình so sánh

Để đánh giá chất lượng phương pháp đề xuất, nhóm sử dụng baseline CNN xây dựng từ đầu và định hướng so sánh với mô hình học chuyển giao (transfer learning).

a) CNN tự xây dựng

Mô hình baseline là CNN xây dựng từ đầu với Keras, gồm:

3 lớp Conv2D (16→32→64) + MaxPooling,

Dense(128) + Softmax(2 lớp).

Baseline được dùng làm tiêu chuẩn tham chiếu để đánh giá hiệu quả của mô hình.

b) Transfer Learning model

Bên cạnh baseline, báo cáo định hướng mở rộng và so sánh với mô hình transfer learning như:

MobileNetV2

ResNet50

EfficientNetB0

Các mô hình này được pretrained trên ImageNet và có thể cải thiện đáng kể kết quả khi dữ liệu y khoa không quá lớn.

7.4. Quy trình đánh giá và đảm bảo tái lập kết quả

Để đảm bảo kết quả thực nghiệm có độ tin cậy và có thể tái lập, nhóm áp dụng các nguyên tắc về đánh giá và quản lý quá trình huấn luyện.

a) Random seed

Trong học sâu, kết quả có thể thay đổi do:

cách khởi tạo trọng số ban đầu,

shuffle dữ liệu,

tính ngẫu nhiên của augmentation.

Vì vậy, nhóm khuyến nghị cố định random seed để kết quả ổn định hơn khi chạy lại mô hình (reproducibility).

b) Lưu log, checkpoint

Trong quá trình training:

lưu lại lịch sử huấn luyện (loss/accuracy curves),

sử dụng checkpoint để lưu mô hình tốt nhất theo validation.

Việc lưu checkpoint giúp:

tránh mất mô hình khi runtime bị ngắt,

đảm bảo luôn có mô hình tốt nhất để evaluate trên test.

c) Cấu trúc folder

Dữ liệu và mô hình được tổ chức theo cấu trúc rõ ràng để dễ quản lý:

dataset/

– train/

– val/

– test/

models/ (lưu model .keras hoặc .h5)

logs/ (lưu lịch sử training nếu cần)

results/ (lưu confusion matrix, hình ảnh dự đoán demo)

Cách tổ chức này giúp:

dễ tái sử dụng,

dễ triển khai demo/inference,

đảm bảo tính khoa học của báo cáo.

VIII. Kết quả thực nghiệm và đánh giá

8.1. Tiêu chí đánh giá

Để đánh giá toàn diện hiệu năng mô hình phân loại ảnh X-quang ngực, báo cáo sử dụng các chỉ số đánh giá phổ biến trong phân loại nhị phân, bao gồm Accuracy, Precision/Recall/F1-score, Confusion Matrix và (tùy chọn) ROC-AUC.

a) Accuracy

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu trong tập đánh giá:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Chỉ số này phản ánh mức độ đúng tổng thể của mô hình. Tuy nhiên, trong trường hợp dữ liệu bị mất cân bằng, Accuracy không phản ánh đầy đủ chất lượng mô hình cho từng lớp.

b) Precision, Recall, F1-score

Precision: mức độ chính xác của các dự đoán dương (dự đoán Pneumonia mà đúng):

$$Precision = \frac{TP}{TP + FP}$$

Recall: khả năng tìm đúng mẫu dương (không bỏ sót Pneumonia):

$$Recall = \frac{TP}{TP + FN}$$

F1-score: trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Các chỉ số này đặc biệt quan trọng trong ảnh y khoa vì:

Recall cao giúp giảm bỏ sót ca bệnh,

Precision cao giúp giảm cảnh báo sai.

c) *Confusion Matrix*

Confusion Matrix thể hiện chi tiết số lượng dự đoán đúng/sai theo từng lớp:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Ma trận này hỗ trợ phân tích loại lỗi mô hình thường gặp (nhầm Normal $\rightarrow$ Pneumonia hay ngược lại).

8.2. *Kết quả quá trình huấn luyện*

Trong quá trình huấn luyện baseline CNN, nhóm theo dõi các chỉ số loss và accuracy theo từng epoch trên hai tập: Train và Validation. Các biểu đồ learning curves giúp đánh giá:

mô hình có hội tụ hay không,

có bị overfitting/underfitting hay không,

độ ổn định quá trình học.

a) *Training/Validation loss curve*

Biểu đồ loss gồm:

`train_loss`: loss trên tập train

`val_loss`: loss trên tập validation

Nhận xét:

Nếu *train_loss giảm* và *val_loss giảm theo* $\rightarrow$ mô hình học tốt.

Nếu *train_loss giảm nhưng val_loss tăng* $\rightarrow$ dấu hiệu overfitting.

b) *Training/Validation accuracy curve*

Biểu đồ accuracy gồm:

`train_accuracy`

`val_accuracy`

Nhận xét:

Nếu accuracy tăng dần và ổn định $\rightarrow$ mô hình hội tụ.

Nếu train cao nhưng val thấp $\rightarrow$ mô hình chưa tổng quát tốt.

8.3. Kết quả trên tập Test

Sau khi huấn luyện xong, mô hình được đánh giá trên tập test để đo khả năng tổng quát hóa đối với dữ liệu chưa từng gặp trong training.

a) Độ chính xác tổng

Kết quả thực nghiệm trên tập test cho thấy:

Test Accuracy $\approx 85\%$

Kết quả này cho thấy baseline CNN có khả năng học được các đặc trưng cơ bản từ ảnh X-quang và đạt hiệu năng khá tốt trong phạm vi bài toán 2 lớp.

b) Classification report

Nhóm sử dụng `classification_report` để đánh giá chi tiết theo từng lớp với các chỉ số:

Precision

Recall

F1-score

Nhận xét chung:

các chỉ số Precision/Recall thể hiện mô hình có thể vẫn gặp khó khăn với một số trường hợp ảnh khó hoặc nhiễu,

F1-score phản ánh sự cân bằng giữa khả năng phát hiện đúng và tránh dự đoán sai.

c) Confusion matrix

Confusion Matrix được xây dựng dựa trên dự đoán của mô hình trên tập test, giúp quan sát trực quan:

số mẫu dự đoán đúng của từng lớp,

số lỗi FP và FN.

Nhận xét:

Nếu FN lớn $\rightarrow$ mô hình có xu hướng bỏ sót ca Pneumonia (rủi ro cao).

Nếu FP lớn $\rightarrow$ mô hình cảnh báo sai (Normal thành Pneumonia).

8.4. Phân tích lỗi (Error Analysis)

Phân tích lỗi là bước quan trọng nhằm hiểu rõ mô hình dự đoán sai trong trường hợp nào và vì sao, từ đó đề xuất hướng cải thiện.

a) Một số ảnh dự đoán đúng/sai tiêu biểu

Nhóm lựa chọn một số mẫu ảnh tiêu biểu trong tập test gồm:

ảnh mô hình dự đoán đúng (Normal/Pneumonia),

ảnh mô hình dự đoán sai.

Với mỗi ảnh, cần minh họa:

nhãn thật (ground truth),

nhãn dự đoán,

xác suất dự đoán (confidence).

b) Nguyên nhân sai (nhiều ảnh, góc chụp, dữ liệu khó)

Các nguyên nhân mô hình dự đoán sai có thể gồm:

Chất lượng ảnh thấp: ảnh mờ, nhiễu, độ tương phản kém.

Biểu hiện bệnh lý không rõ ràng: vùng tổn thương mờ nhạt, khó phân biệt với Normal.

Góc chụp/điều kiện chụp khác nhau: làm thay đổi độ sáng và cấu trúc ảnh.

Dataset bias: dữ liệu từ các nguồn khác nhau (domain shift) khiến mô hình khó tổng quát.

Mất cân bằng dữ liệu: mô hình dễ thiên về lớp chiếm đa số (Pneumonia).

c) Nhận xét và bài học rút ra

Từ kết quả và lỗi quan sát được, có thể rút ra:

Baseline CNN đạt kết quả tương đối tốt nhưng vẫn có hạn chế với các ảnh khó/nhiều.

Cần bổ sung các kỹ thuật tăng cường dữ liệu, regularization, hoặc xử lý imbalance tốt hơn.

Transfer Learning là hướng cải tiến hợp lý giúp mô hình học đặc trưng mạnh hơn, giảm overfitting và tăng accuracy.

IX. Kết luận và hướng phát triển

9.1. Kết luận

Trong báo cáo này, nhóm đã xây dựng và đánh giá thành công một mô hình học sâu dựa trên *mạng nơ-ron tích chập (CNN)* nhằm giải quyết bài toán *phân loại ảnh X-quang ngực nhị phân (Normal/Pneumonia)*. Toàn bộ pipeline được triển khai trên môi trường Google Colab bằng thư viện *TensorFlow/Keras*, bao gồm các bước: chuẩn bị dữ liệu, tiền xử lý, xây dựng mô hình, huấn luyện và đánh giá trên tập kiểm tra.

Mô hình baseline CNN được thiết kế theo kiến trúc tuần tự với ba khối tích chập (Conv2D) kết hợp pooling, sau đó là các lớp Dense cho mục tiêu phân loại hai lớp. Kết quả thực nghiệm cho thấy mô hình đạt:

Accuracy trên tập test: 85%

Precision (macro): 0.88

Recall (macro): 0.81

F1-score (macro): 0.83

Ngoài ra, kết quả confusion matrix và classification report cho thấy mô hình có khả năng phát hiện lớp Pneumonia khá tốt (Recall cao), tuy nhiên vẫn còn nhầm lẫn đáng kể đối với một số trường hợp thuộc lớp Normal. Nhìn chung, mô hình đã chứng minh tính khả thi của việc áp dụng CNN trong phân loại ảnh X-quang và đóng vai trò như một nền tảng ban đầu cho các hệ thống hỗ trợ phân tích ảnh y khoa.

9.2. Hạn chế của đề tài

Mặc dù đạt được kết quả khả quan, đề tài vẫn còn một số hạn chế như sau:

Hiệu năng chưa cao ở lớp Normal:

Theo classification report, lớp Normal có recall thấp (≈ 0.45), cho thấy mô hình vẫn nhầm lẫn nhiều ảnh Normal thành Pneumonia. Điều này có thể gây cảnh báo sai (false positive) trong ứng dụng thực tế.

Dữ liệu mất cân bằng:

Số lượng ảnh Pneumonia lớn hơn Normal dẫn tới hiện tượng mô hình thiên lệch về lớp Pneumonia, làm giảm hiệu quả nhận diện lớp Normal.

Mô hình baseline còn đơn giản:

CNN xây dựng từ đầu với số lớp tích chập hạn chế nên khả năng trích xuất đặc trưng nâng cao chưa mạnh bằng các mô hình pretrained hiện đại.

Chưa sử dụng các kỹ thuật cải thiện huấn luyện:

Notebook chưa áp dụng đầy đủ các kỹ thuật nâng cao như:

- data augmentation chuyên sâu,
- class weight,
- EarlyStopping/ModelCheckpoint tối ưu,
- learning rate scheduler.

Khả năng giải thích mô hình chưa được đề cập:

Trong ảnh y khoa, việc mô hình dự đoán cần minh bạch. Tuy nhiên đề tài chưa triển khai các phương pháp giải thích như Grad-CAM để trực quan hóa vùng mô hình chú ý.

9.3. Hướng phát triển

Để nâng cao chất lượng và tính ứng dụng của hệ thống, đề tài có thể tiếp tục phát triển theo các hướng sau:

Cải thiện dữ liệu và tiền xử lý

Tăng cường dữ liệu (augmentation) theo hướng phù hợp ảnh X-quang: rotation nhỏ, contrast adjustment, zoom/shift.

Áp dụng xử lý mất cân bằng như *class weight* hoặc oversampling lớp Normal.

Nâng cấp mô hình theo hướng Transfer Learning

Sử dụng backbone pretrained như *MobileNetV2*, *ResNet50*, *EfficientNetB0* để trích xuất đặc trưng mạnh hơn.

Fine-tuning các block cuối của backbone nhằm tăng độ chính xác và giảm overfitting.

Tối ưu chiến lược huấn luyện

Sử dụng *EarlyStopping* + *ModelCheckpoint* để chọn mô hình tốt nhất.

Áp dụng *learning rate scheduler* (*ReduceLROnPlateau*) để tăng khả năng hội tụ.

Thử nghiệm nhiều batch size, learning rate, dropout để tìm cấu hình tối ưu.

Bổ sung đánh giá nâng cao

Tính ROC-AUC và PR-AUC để đánh giá mô hình tốt hơn trong trường hợp mất cân bằng.

Thực hiện phân tích lỗi sâu (error analysis) với các trường hợp dự đoán sai điển hình.

Tăng tính giải thích và triển khai ứng dụng

Triển khai *Grad-CAM* để minh họa vùng ảnh khiến mô hình đưa ra dự đoán.

Xây dựng demo ứng dụng đơn giản bằng *Streamlit/Gradio*, cho phép người dùng tải ảnh X-quang và nhận kết quả dự đoán cùng xác suất.

X. Tài liệu tham khảo

- [1] P. Mooney, “Chest X-Ray Images (Pneumonia),” Kaggle Dataset. [Online]. Available: *Kaggle*.
- [2] D. S. Kermany, K. Zhang, and M. Goldbaum, “Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification,” *Mendeley Data*, 2018.
- [3] TensorFlow, “tf.keras.preprocessing.image_dataset_from_directory,” *TensorFlow API Documentation*.
- [4] Keras, “Image data loading: image_dataset_from_directory,” *Keras Documentation*.
- [5] TensorFlow, “Convolutional Neural Network (CNN) tutorial,” *TensorFlow Tutorials*.
- [6] TensorFlow, “Load and preprocess images,” *TensorFlow Tutorials*.
- [7] L. M. Huy, *Tài liệu môn học Học sâu (Deep Learning) – Bài giảng và slide môn học*, Trường Đại học Phenikaa, Hà Nội, Việt Nam, 2025–2026.
- [8] N. V. Tới, *Tài liệu môn học Xử lý ảnh (Image Processing) – Bài giảng và slide môn học*, Trường Đại học Phenikaa, Hà Nội, Việt Nam, 2025–2026.

XI. Phụ lục

A. Link mã nguồn (GitHub/Drive)

Dataset (Google Drive):

https://drive.google.com/file/d/1fSVPx8n_34hz2e81-EM2NZUwUihaZIlv/view?usp=sharing

Mã nguồn (GitHub):

<https://github.com/trdieu/Github/tree/master/Deep%20Learning>

Notebook huấn luyện (Google Colab):

https://colab.research.google.com/drive/1LzVBeNe0kHQBlorKpJa8_fNRAPMHziIV?usp=sharing

B. Cấu hình huấn luyện chi tiết

Nền tảng: Google Colab

Framework: TensorFlow/Keras

Bộ dữ liệu: Chest X-ray Pneumonia (Normal/Pneumonia)

Kích thước ảnh đầu vào: $180 \times 180 \times 3$

Tiền xử lý:

Resize ảnh về 180×180

Rescaling pixel: $X = X/255$

Mô hình sử dụng (Baseline CNN):

Rescaling($1./255$)

Conv2D(16, 3×3 , ReLU) + MaxPooling

Conv2D(32, 3×3 , ReLU) + MaxPooling

Conv2D(64, 3×3 , ReLU) + MaxPooling

Flatten

Dense(128, ReLU)

Dense(2, Softmax)

Optimizer: Adam

Loss function: Cross Entropy (phù hợp phân loại 2 lớp với Softmax)

Metrics: Accuracy

Số epochs: 10

Kết quả test (tóm tắt):

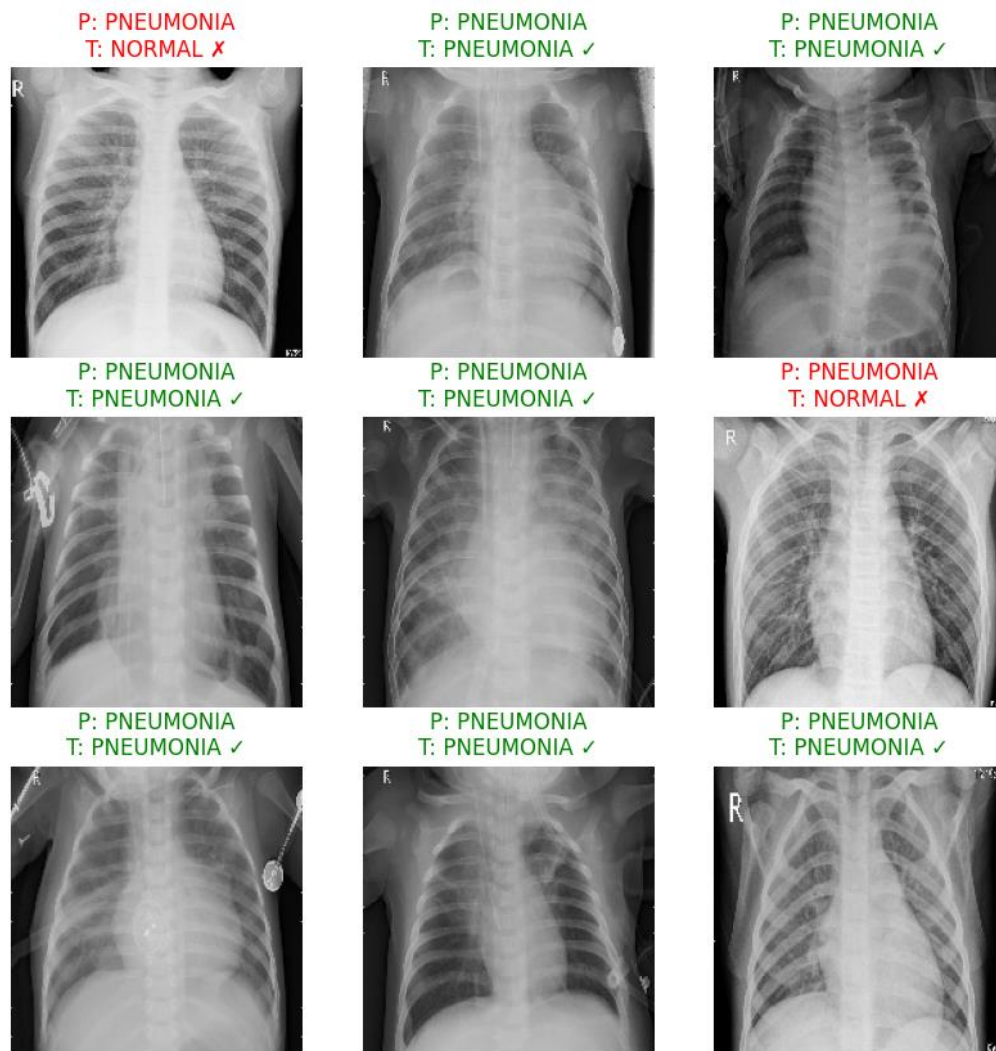
Accuracy ≈ 0.8542

Precision (macro) ≈ 0.88

Recall (macro) ≈ 0.81

F1-score (macro) ≈ 0.83

C. Một số hình ảnh minh họa & kết quả dự đoán



XII. Lời cảm ơn

Em xin gửi lời cảm ơn chân thành đến TS. Nguyễn Văn Tới – giảng viên môn Xử lý ảnh, Trường Đại học Phenikaa – đã tận tình giảng dạy, cung cấp tài liệu học tập và hướng dẫn trong suốt quá trình thực hiện đề tài. Những kiến thức nền tảng và góp ý của thầy đã giúp em hoàn thiện bài báo cáo, hiểu rõ hơn về quy trình xây dựng, huấn luyện và đánh giá mô hình học sâu trên dữ liệu ảnh.

Em xin chân thành cảm ơn thầy.